COALA
EXECUTION WALKTHROUGH

50+ PAGE
SOURCE-LEVEL TRACE

Customer support triage - iterative reason-act-observe

A complete source-level trace using the exact use-case route support-iterative and the sample goal shown in this document.

This guide follows the request through the HTTP boundary, request validation, session creation or resumption, action registration, context construction, the configured decision procedure, every cycle, and the serialized result. The document intentionally treats the offline scripted model as deterministic fixtures, not as an LLM.

Use case: Customer support triage - iterative reason-act-observe
Based on the supplied project source tree. Code excerpts identify source file and line ranges. Simulated provider results are labeled as such.

Purpose: follow every call from HTTP arrival through each decision cycle to the final response, and understand what every significant method does with its inputs and state.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

How to use this walkthrough

Read the method-call ledger top to bottom. A call is listed only when the current implementation makes it, while framework dispatch supplied by Spring is labeled as framework behavior rather than invented application code. Pages that show source excerpts cite the project-relative file and line span so you can open the same implementation in the ZIP.

Each cycle is traced from the latest observation through planning, selected action validation, execution, working-memory updates, event creation, and the check that determines whether the loop continues. If a cycle does not call the language model, the guide says so explicitly.

Code-level interpretation

- “Scripted fixture” means a hard-coded deterministic response chosen by operation marker, cycle number, or simple string checks.
- “Working-memory write” means a key/value update to the per-run WorkingMemory object, not a durable database commit.
- “Long-term memory” here is a session-scoped in-process store; it survives later requests using the returned session ID only while the application process and session remain alive.

Actual source excerpt: dev/coala/core/procedure/AbstractLoopingDecisionProcedure.java (source lines 25-45)

```
 */
public abstract class AbstractLoopingDecisionProcedure implements CoalaDecisionProcedure {
    public static final int MAX_GOAL_CHARACTERS = 4_000;

    @Override
    public final AgentRunResult run(String goal, AgentContext context, ActionRegistry actions, int maxCycles) {
        if (goal == null || goal.isBlank()) throw new IllegalArgumentException("goal must not be blank");
        if (goal.length() > MAX_GOAL_CHARACTERS) throw new IllegalArgumentException("goal must be no more than "
         + MAX_GOAL_CHARACTERS + " characters");
        goal = goal.trim();
        if (maxCycles < 1 || maxCycles > 100) throw new IllegalArgumentException("maxCycles must be between 1
        and 100");

        context.workingMemory().put("goal", goal.trim());
        context.workingMemory().put("latest_observation", "User request received: " + goal.trim());
        List<DecisionEvent> events = new ArrayList<>();
        String finalObservation = "";

        for (int cycle = 1; cycle <= maxCycles; cycle++) {
            context.setCycle(cycle);
            String observation = context.workingMemory().getString("latest_observation", "");
            events.add(new DecisionEvent(cycle, DecisionStage.OBSERVATION,
                    truncate(observation, 240)));
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The exact incoming request

The walkthrough assumes the request body below, omits maxCycles so the service uses the configured default (normally 8), and omits sessionId so the server creates a new opaque session. This is a deliberate first-run path: it lets the trace show session creation before context construction.

The user goal is: My account is locked and I see an unauthorized payment; help me regain access safely.. The goal remains the task text in working memory; the framework does not convert it into a typed domain object before the selected decision procedure reasons about it.

State and observable values

Field	Value / meaning
HTTP method/path	POST /api/use-cases/support-iterative/run
Request JSON	{"goal":"My account is locked and I see an unauthorized payment; help me regain access safely."}
maxCycles	null in JSON; service default is injected from coala.run.max-cycles (default 8)
sessionId	omitted; SessionMemoryStore creates a UUID session

Actual source excerpt: dev/coala/api/RunRequest.java (source lines 1-15)

```
package dev.coala.api;

public record RunRequest(String goal, Integer maxCycles, String sessionId) {
    public RunRequest {
        if (goal == null || goal.isBlank()) throw new IllegalArgumentException("goal must not be blank");
        if (goal.length() > 4_000) throw new IllegalArgumentException("goal must be no more than 4000
        characters");
        if (maxCycles != null && (maxCycles < 1 || maxCycles > 100)) {
            throw new IllegalArgumentException("maxCycles must be between 1 and 100");
        }
        if (sessionId != null && !sessionId.isBlank()
                && !sessionId.trim().matches("[A-Za-z0-9._-]{1,64}")) {
            throw new IllegalArgumentException("sessionId must be 1-64 characters using letters, digits, '.',
        '_' or '-'.");
        }
    }
}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

What this example does - and does not - prove

The use case terminates in 2 decision cycles by executing escalate_case in cycle 2. It demonstrates the mechanics of the configured decision procedure and action boundary, not the factual correctness of a production integration.

A safe escalation recommendation that tells the user to use the provider verified support channel. SUP-DEMO-1042 is fictional. There is no ticket creation, identity verification, account lookup, or payment reversal.

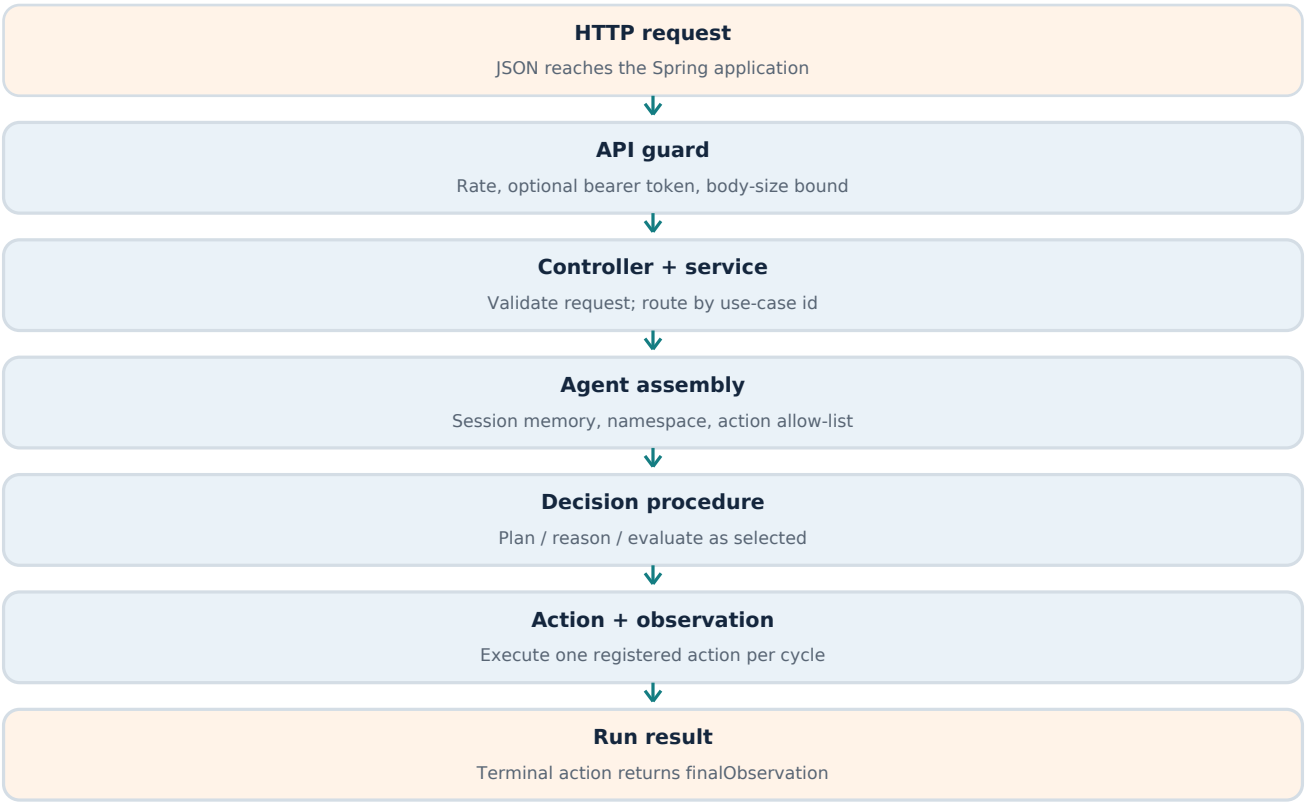
No real search provider, airline, hotel, calendar, ticketing platform, identity service, or financial system is called by this demo path. The simulated results are appropriate for explaining control flow, not for making real-world decisions.

Important boundary
A successful terminal action only means that the action returned terminal=true. It does not mean the external-world task was performed by a real provider.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

A single visual map of the full call chain

This is the route for this use case. It is a call map, not a new CoALA architectural component. The API guard is application infrastructure; the decision procedure, memory, action execution, and observation loop are the framework side of the route.



Important boundary
The arrows mean “control passes to”; they do not mean every box is a Java method. Some steps are framework dispatch or a group of calls explained on subsequent pages.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The terminology used in the call ledger

A decision cycle is one iteration of AbstractLoopingDecisionProcedure.run: set the cycle number, record the current observation, clear stale plan-selection keys, ask the configured procedure for one next action, validate it, execute it, write its result as the latest observation, then stop or continue.

A reasoning call is a call through ReasoningAction.reason to the configured LanguageModel. Different procedures make different numbers of reasoning calls per cycle. The deliberative procedure usually reasons once for candidate proposal and once for evaluation; the plan-first procedure reasons to make the plan and can then reuse stored plan steps without another model call; the iterative procedure reasons once per cycle.

A selected action is a structured ActionCall with an actionId and string arguments. It is not executable code generated by the model. The ActionRegistry resolves it to a pre-registered implementation and validates its declared argument contract.

Actual source excerpt: dev/coala/core/PlanDecision.java (source lines 1-31)

```
package dev.coala.core;

import java.util.List;
import java.util.Objects;

/** Result of planning one next action, including structured candidates/evaluations when available. */
public record PlanDecision(
        ActionCall selectedAction,
        List<ActionCall> candidates,
        List<Double> evaluationScores,
        List<DecisionEvent> events) {

    public PlanDecision {
        Objects.requireNonNull(selectedAction, "selectedAction");
        candidates = candidates == null || candidates.isEmpty() ? List.of(selectedAction) :
        List.copyOf(candidates);
        evaluationScores = evaluationScores == null ? List.of() : List.copyOf(evaluationScores);
        events = events == null ? List.of() : List.copyOf(events);
        if (!candidates.contains(selectedAction)) {
            throw new IllegalArgumentException("selectedAction must be present in the planned candidate list");
        }
        if (!evaluationScores.isEmpty() && evaluationScores.size() != candidates.size()) {
            throw new IllegalArgumentException("evaluationScores must be empty or contain one score per
        candidate");
        }
        if (evaluationScores.stream().anyMatch(score -> score == null || !Double.isFinite(score))) {
            throw new IllegalArgumentException("evaluation scores must be finite numbers");
        }
    }

    /** Direct/iterative decision procedures may propose one candidate without explicit evaluation. */
    public PlanDecision(ActionCall selectedAction, List<DecisionEvent> events) {
        this(selectedAction, List.of(selectedAction), List.of(), events);
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

HTTP entry: the request as received by the app guard

The first application-owned method for an /api request is ApiGuardFilter.doFilterInternal, inherited from Spring's OncePerRequestFilter. The framework calls it in the servlet filter chain. It checks rate-limit allowance first; if a bearer token is configured, it validates Authorization; then it bounds and buffers the request body. Only after those checks does it invoke filterChain.doFilter.

For this sample, the default server address is loopback and the token is blank in the local demo configuration, so the bearer-token branch is disabled. The rate limit and maximum request-byte checks still exist. A deployment bound to a non-loopback address fails construction unless COALA_API_TOKEN is configured.

Method-call ledger

01	OncePerRequestFilter dispatches to shouldNotFilter(request).
02	shouldNotFilter checks URI begins with contextPath + /api/.
03	doFilterInternal calls allowRate(remoteAddress).
04	If configuredToken is nonempty, validBearerToken(Authorization) runs.
05	enforceRequestSize reads at most maxRequestBytes + 1 and may reject oversized bodies.
06	filterChain.doFilter(boundedRequest, response) passes control to Spring MVC.

Actual source excerpt: dev/coala/api/ApiGuardFilter.java (source lines 62-96)

```
    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String apiPrefix = request.getContextPath() + "/api/";
        return !request.getRequestURI().startsWith(apiPrefix);
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain
        filterChain)
            throws ServletException, IOException {
        if (!allowRate(request.getRemoteAddr())) {
            writeProblem(response, HttpServletResponse.SC_TOO_MANY_REQUESTS,
                    "Rate limit exceeded", "Too many API requests from this client. Retry after the current
        minute window.");
            return;
        }

        // Authenticate before consuming a request body. A rejected caller should not
        // be able to force the application to buffer up to the request-size limit.
        if (configuredToken.length > 0 && !validBearerToken(request.getHeader("Authorization"))) {
            response.setHeader("WWW-Authenticate", "Bearer");
            writeProblem(response, HttpServletResponse.SC_UNAUTHORIZED,
                    "Unauthorized", "A valid bearer token is required for this API.");
            return;
        }

        HttpServletRequest boundedRequest;
        try {
            boundedRequest = enforceRequestSize(request, response);
        } catch (IOException ex) {
            writeProblem(response, HttpServletResponse.SC_BAD_REQUEST,
                    "Invalid request body", "The request body could not be read.");
            return;
        }
        if (boundedRequest == null) return;
        filterChain.doFilter(boundedRequest, response);
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The rate-limit branch is independent from CoALA

allowRate keys the in-memory request counter by remote address and uses a 60-second monotonic-time window. A new address gets a bucket with request count one; a current bucket is accepted only while its count is below requestsPerMinute; old buckets are pruned periodically. This is a small application-shell guard, not a cognitive action and not part of the decision procedure.

If the bucket map is full and cannot add a new client bucket, the filter rejects rather than growing its map without bound. A rejected request receives an application/problem+json response and does not reach the controller. That means there will be no session, context, memory retrieval, model call, or decision cycle for a rejected request.

Actual source excerpt: dev/coala/api/ApiGuardFilter.java (source lines 104-125)

```
    private boolean allowRate(String remoteAddress) {
        String key = remoteAddress == null || remoteAddress.isBlank() ? "unknown" : remoteAddress;
        long now = System.nanoTime();
        AtomicBoolean accepted = new AtomicBoolean(false);
        buckets.compute(key, (ignored, current) -> {
            if (current == null && buckets.size() >= MAX_RATE_LIMIT_BUCKETS) {
                accepted.set(false);
                return null;
            }
            if (current == null || now - current.windowStartedAt >= WINDOW_NANOS) {
                accepted.set(true);
                return new RateBucket(now, 1);
            }
            if (current.requests >= requestsPerMinute) return current;
            accepted.set(true);
            return new RateBucket(current.windowStartedAt, current.requests + 1);
        });

        if (requestCounter.incrementAndGet() % 256 == 0 || buckets.size() > MAX_RATE_LIMIT_BUCKETS) {
            pruneRateBuckets(now);
        }
        return accepted.get();
```

Important boundary
The in-memory rate limiter is process-local. A clustered deployment would need a shared rate-limit implementation, but the framework’s CoALA decision-cycle semantics would not change.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The request-size guard and replayable body wrapper

enforceRequestSize only buffers methods that may carry a body (POST, PUT, PATCH). It first rejects a declared Content-Length over the configured limit, then reads no more than maxRequestBytes + 1 bytes so chunked bodies without a useful Content-Length are still bounded. If the body is too large, it sends 413 and returns null; doFilterInternal returns without calling the next filter.

For a bounded body, CachedBodyRequestWrapper wraps the original request so the dispatcher and JSON converter can read the bytes after the guard has already consumed them. The wrapper constructs a new ByteArrayInputStream for each getInputStream call. This keeps the guard from breaking normal JSON deserialization.

Code-level interpretation

- A request larger than the limit exits before CoalaController.run.
- A well-sized request is handed forward through filterChain.doFilter, not directly to the controller by this method.

Actual source excerpt: dev/coala/api/ApiGuardFilter.java (source lines 128-145)

```
    private HttpServletRequest enforceRequestSize(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
        String method = request.getMethod();
        boolean mayHaveBody = "POST".equalsIgnoreCase(method) || "PUT".equalsIgnoreCase(method)
                || "PATCH".equalsIgnoreCase(method);
        if (!mayHaveBody) return request;
        if (request.getContentLengthLong() > maxRequestBytes) {
            writeProblem(response, HttpServletResponse.SC_REQUEST_ENTITY_TOO_LARGE,
                    "Request too large", "The request body exceeds the configured size limit.");
            return null;
        }
        byte[] body = request.getInputStream().readNBytes(maxRequestBytes + 1);
        if (body.length > maxRequestBytes) {
            writeProblem(response, HttpServletResponse.SC_REQUEST_ENTITY_TOO_LARGE,
                    "Request too large", "The request body exceeds the configured size limit.");
            return null;
        }
        return new CachedBodyRequestWrapper(request, body);
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Spring MVC dispatch reaches CoalaController.run

After the filter passes the wrapped request, Spring MVC matches POST /api/use-cases/{id}/run to CoalaController.run. Spring binds the path segment to id and deserializes the JSON body into RunRequest. No application code in the agent core manually parses the HTTP body.

The controller itself is deliberately thin: it delegates use-case semantics to DemoUseCaseService. It does not know which cognitive procedure is selected, which actions are registered, or how working memory is managed.

Method-call ledger

01	Spring route matches @RequestMapping("/api") + @PostMapping("/use-cases/{id}/run").
02	Jackson creates RunRequest from JSON.
03	CoalaController.run(id, request) reads request.goal(), request.maxCycles(), request.sessionId().
04	useCases.run(...) performs application-specific routing.

Actual source excerpt: dev/coala/api/CoalaController.java (source lines 17-29)

```
public class CoalaController {
    private final DemoUseCaseService useCases;

    public CoalaController(DemoUseCaseService useCases) { this.useCases = useCases; }

    @GetMapping("/use-cases")
    public List<DemoUseCaseDescriptor> listUseCases() { return useCases.list(); }

    @PostMapping("/use-cases/{id}/run")
    public UseCaseRunResponse run(@PathVariable String id, @RequestBody RunRequest request) {
        return useCases.run(id, request.goal(), request.maxCycles(), request.sessionId());
    }
}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

RunRequest performs early input validation

RunRequest is a Java record, so its compact constructor runs when the JSON converter constructs the record. The goal cannot be null or blank and cannot exceed 4,000 characters. maxCycles, if present, must be between 1 and 100. sessionId, if present and nonblank, is restricted to a short safe alphabet and 64 characters.

These checks are distinct from the AbstractLoopingDecisionProcedure validation. The DTO protects the HTTP boundary; the procedure repeats critical goal and cycle bounds because it is a reusable core entry point that may also be called without HTTP.

Actual source excerpt: dev/coala/api/RunRequest.java (source lines 1-15)

```
package dev.coala.api;

public record RunRequest(String goal, Integer maxCycles, String sessionId) {
    public RunRequest {
        if (goal == null || goal.isBlank()) throw new IllegalArgumentException("goal must not be blank");
        if (goal.length() > 4_000) throw new IllegalArgumentException("goal must be no more than 4000
        characters");
        if (maxCycles != null && (maxCycles < 1 || maxCycles > 100)) {
            throw new IllegalArgumentException("maxCycles must be between 1 and 100");
        }
        if (sessionId != null && !sessionId.isBlank()
                && !sessionId.trim().matches("[A-Za-z0-9._-]{1,64}")) {
            throw new IllegalArgumentException("sessionId must be 1-64 characters using letters, digits, '.',
        '_' or '-'.");
        }
    }
}
```

Important boundary
Input schema validation does not establish that the user is authorized for the content of a goal. Demo session IDs are capabilities for demo memory resumption, not a substitute for application authentication.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Invalid requests and internal failures take different paths

ApiExceptionHandler maps malformed JSON and validation-related IllegalArgumentException to a 400 response; a missing or expired session is 404; provider failures are 502; invalid model output is 502; action execution failures and planning failures become sanitized 500 responses. The response avoids provider URLs, prompts, credentials, and nested exception details.

The exception boundary matters for the trace: if an action fails, AbstractLoopingDecisionProcedure wraps runtime failures as ActionExecutionException; CoalaAgent publishes a generic run-failed trace event to listeners and rethrows; the API advice converts that exception to a stable response. In this example the deterministic fixtures finish normally, so this error path is not taken.

Actual source excerpt: dev/coala/api/ApiExceptionHandler.java (source lines 18-78)

```
@ExceptionHandler(IllegalArgumentException.class)
ProblemDetail badRequest(IllegalArgumentException ex) {
    ProblemDetail detail = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, ex.getMessage());
    detail.setTitle("Invalid request");
    return detail;
}

@ExceptionHandler(HttpMessageNotReadableException.class)
ProblemDetail unreadableRequest(HttpMessageNotReadableException ex) {
    ProblemDetail detail = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, "The request body must be valid JSON and satisfy the request schema.");
    detail.setTitle("Invalid request body");
    return detail;
}

@ExceptionHandler(NoSuchElementException.class)
ProblemDetail sessionNotFound(NoSuchElementException ex) {
    ProblemDetail detail = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    detail.setTitle("Session not found");
    return detail;
}

@ExceptionHandler(LanguageModelException.class)
ProblemDetail languageModelFailure(LanguageModelException ex) {
    // Do not expose provider URLs, credentials, response bodies, or nested exception details.
    ProblemDetail detail = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_GATEWAY, "The configured language-model provider could not complete the
    request.");
    detail.setTitle("Language-model provider failure");
    return detail;
}

@ExceptionHandler(InvalidModelOutputException.class)
ProblemDetail invalidModelOutput(InvalidModelOutputException ex) {
    ProblemDetail detail = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_GATEWAY, "The language-model response did not satisfy the required
    structured-output contract.");
    detail.setTitle("Invalid language-model response");
    return detail;
}

@ExceptionHandler(ActionExecutionException.class)
ProblemDetail actionExecutionFailure(ActionExecutionException ex) {
    ProblemDetail detail = ProblemDetail.forStatusAndDetail(
            HttpStatus.INTERNAL_SERVER_ERROR, "The selected action failed while executing.");
    detail.setTitle("Action execution failed");
    return detail;
}

@ExceptionHandler(PlanningExhaustedException.class)
ProblemDetail planningExhausted(PlanningExhaustedException ex) {
    ProblemDetail detail = ProblemDetail.forStatusAndDetail(
            HttpStatus.INTERNAL_SERVER_ERROR, "The agent could not select an action within its configured
    planning-attempt limit.");
    detail.setTitle("Planning exhausted");
    return detail;
}

@ExceptionHandler(DecisionProcedureException.class)
ProblemDetail decisionFailure(DecisionProcedureException ex) {
    // This is an application/agent failure, not a user input validation error.
    ProblemDetail detail = ProblemDetail.forStatusAndDetail(
            HttpStatus.INTERNAL_SERVER_ERROR, "The agent decision procedure could not complete the
    request.");
    detail.setTitle("Agent execution failed");
```

Important boundary
The report traces the successful simulation. Error paths are described separately, not silently substituted into the successful cycle ledger.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

DemoUseCaseService.run is the use-case router

DemoUseCaseService.run first checks the ID against the explicit allow-list of six supported demos. It chooses maxCycles from the request or configured default, validates that bound, and acquires the requested/new session. A switch expression then dispatches to the case-specific builder: research uses runResearch, travel uses runTravel, and support uses runSupport.

This switch is where use-case-specific wiring lives. The reusable procedures do not know what “Paris,” a research source, or a support ticket means. They only see a bounded goal, a working-memory context, an action registry, and a procedure-specific planning algorithm.

Method-call ledger

01	Check useCaseId against six allowed strings.
02	Compute boundedCycles = request maxCycles or defaultMaxCycles.
03	sessionMemoryStore.acquire(requestedSessionId).
04	Dispatch to runResearch / runTravel / runSupport for this guide.
05	Wrap result with new UseCaseRunResponse(session.sessionId(), result).

Actual source excerpt: dev/coala/demo/DemoUseCaseService.java (source lines 78-97)

```
public UseCaseRunResponse run(String useCaseId, String goal, Integer maxCycles, String requestedSessionId) {
    if (!List.of("dialogue-direct", "coding-iterative", "puzzle-deliberative",
            "research-deliberative", "travel-plan-first", "support-iterative").contains(useCaseId)) {
        throw new IllegalArgumentException("Unknown use case id: " + useCaseId);
    }
    int boundedCycles = maxCycles == null ? defaultMaxCycles : maxCycles;
    if (boundedCycles < 1 || boundedCycles > 100) throw new IllegalArgumentException("maxCycles must be
    between 1 and 100");
    SessionMemoryStore.Session session = sessionMemoryStore.acquire(requestedSessionId);
    AgentRunResult result = switch (useCaseId) {
        case "dialogue-direct" -> runAgent(useCaseId, direct,
                List.of(new DialogueReplyAction()), goal, boundedCycles, session);
        case "coding-iterative" -> runCoding(goal, boundedCycles, session);
        case "puzzle-deliberative" -> runPuzzle(goal, boundedCycles, session);
        case "research-deliberative" -> runResearch(goal, boundedCycles, session);
        case "travel-plan-first" -> runTravel(goal, boundedCycles, session);
        case "support-iterative" -> runSupport(goal, boundedCycles, session);
        default -> throw new IllegalArgumentException("Unknown use case id: " + useCaseId);
    };
    return new UseCaseRunResponse(session.sessionId(), result);
}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The first request creates a session; later requests must resume it

SessionMemoryStore.acquire returns createSession when requestedSessionId is null or blank. createSession generates a random UUID not selected by the caller, seeds an InMemoryLongTermMemory with three demo records, stores the session in a bounded LRU map, and returns a Session containing the id and memory reference. If a nonblank ID is provided, acquire only resumes an existing unexpired entry; it does not let the caller invent or reserve a chosen ID.

For this walkthrough, assume no session ID. The session ID appears in the response envelope, and a later request can send it back to resume long-term memory. WorkingMemory is different: AgentContextFactory creates a fresh WorkingMemory for each run.

Actual source excerpt: dev/coala/demo/SessionMemoryStore.java (source lines 48-94)

```
            throw new IllegalArgumentException("sessionId must be 1-64 characters using letters, digits, '.',
        '_' or '-'.");
        }

        long now = System.nanoTime();
        evictExpired(now);
        Entry entry = sessions.get(sessionId);
        if (entry == null) {
            throw new NoSuchElementException("Session not found or expired. Start a new session without
        providing a sessionId.");
        }
        entry.lastAccessNanos = now;
        return new Session(sessionId, entry.memory);
    }

    /** Creates a new session ID generated by the server; callers cannot choose or reserve an ID. */
    public synchronized Session createSession() {
        long now = System.nanoTime();
        evictExpired(now);
        String sessionId;
        do {
            sessionId = UUID.randomUUID().toString();
        } while (sessions.containsKey(sessionId));
        Entry entry = new Entry(createSeededMemory(), now);
        sessions.put(sessionId, entry);
        evictExcessSessions();
        return new Session(sessionId, entry.memory);
    }

    public synchronized int activeSessionCount() {
        evictExpired(System.nanoTime());
        return sessions.size();
    }

    private void evictExpired(long now) {
        Iterator<Map.Entry<String, Entry>> iterator = sessions.entrySet().iterator();
        while (iterator.hasNext()) {
            Entry entry = iterator.next().getValue();
            if (now - entry.lastAccessNanos >= idleTtlNanos) iterator.remove();
        }
    }

    private void evictExcessSessions() {
        Iterator<Map.Entry<String, Entry>> iterator = sessions.entrySet().iterator();
        while (sessions.size() > maxSessions && iterator.hasNext()) {
            iterator.next();
            iterator.remove(); // LRU eviction; in-flight runs retain their local memory reference.
        }
    }
```

Important boundary
The store is in-process memory. A restart or eviction loses the demo session, so it should not be described as durable production persistence.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Support uses a scoped namespace but its iterative planner does not retrieve

runSupport creates MemoryNamespace("support-iterative", sessionId, "demo", "support") and registers classify_support_case, suggest_safe_steps, and escalate_case. IterativeReasonActDecisionProcedure reasons from the goal and latest observation, but it does not call MemoryRetrievalAction; this is a deliberately small demonstration of a repeated reason-act-observe loop.

The classify action puts a compact support.triage object into working memory. The next scripted reasoning call is selected from the goal text and the available action catalog. It is important not to claim that the scripted fixture reads the triage object unless its actual code does so: the fixture does not.

State and observable values

Field	Value / meaning
agentId	support-iterative
domain	support
registered actions	classify, safe steps, escalate
retrieval this run	none

Actual source excerpt: dev/coala/demo/DemoUseCaseService.java (source lines 137-143)

```
    private AgentRunResult runSupport(String goal, int maxCycles, SessionMemoryStore.Session session) {
        MemoryNamespace namespace = namespace("support-iterative", session.sessionId(), "support");
        return runAgent("support-iterative", iterative, List.of(
                SupportTriageActions.classifyCase(),
                SupportTriageActions.suggestSafeSteps(),
                SupportTriageActions.escalateCase()), goal, maxCycles, session, namespace);
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The selected case wires actions into a reusable procedure

The use-case builder calls runAgent with agent id support-iterative, procedure iterative-reason-act, its six/four/three registered actions, the goal, max-cycle limit, session and namespace. The action list defines the allow-list: the model can propose only IDs in that list, and ActionRegistry resolves each ID to the registered object.

For research, the action instances are WebSearchGroundingAction backed by SimulatedWebSearchProvider, formatEvidenceBrief, EpisodicMemoryLearningAction, and deliverResearchBrief. For travel, the six action IDs correspond to the six plan steps. For support, three action IDs support classification and two possible terminal outcomes.

State and observable values

Field	Value / meaning
procedure	case-specific selection
action registry	constructed from only the actions passed to runAgent
context factory	creates fresh WorkingMemory and resolves this session memory
max cycles	request/configured maximum; run terminates earlier if action is terminal

Actual source excerpt: dev/coala/demo/DemoUseCaseService.java (source lines 145-168)

```
    private AgentRunResult runAgent(String agentId, CoalaDecisionProcedure procedure,
                                    List<? extends CoalaAction> actions, String goal, int maxCycles,
                                    SessionMemoryStore.Session session) {
        return runAgent(agentId, procedure, actions, goal, maxCycles, session, MemoryNamespace.DEFAULT);
    }

    private AgentRunResult runAgent(String agentId, CoalaDecisionProcedure procedure,
                                    List<? extends CoalaAction> actions, String goal, int maxCycles,
                                    SessionMemoryStore.Session session, MemoryNamespace namespace) {
        AgentContextFactory contextFactory = AgentContextFactory.scoped(request -> session.longTermMemory());
        CoalaAgentDefinition definition = CoalaAgentDefinition.of(agentId, procedure, actions, contextFactory,
        maxCycles);
        CoalaAgent agent = new CoalaAgent(definition);
        return agent.run(new AgentRunRequest(goal, session.sessionId(), "demo", namespace));
    }

    private static MemoryNamespace namespace(String agentId, String sessionId, String domain) {
        return new MemoryNamespace(agentId, sessionId, "demo", domain);
    }

    private static void ensureSeed(SessionMemoryStore.Session session, MemoryNamespace namespace,
                                   String seedId, String content) {
        boolean exists = session.longTermMemory().list(namespace, 1_000).stream()
                .anyMatch(record -> seedId.equals(record.metadata().get("demoSeed")));
        if (!exists) {
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

runAgent creates context policy and the runtime facade

runAgent creates AgentContextFactory.scoped(request -> session.longTermMemory()). On each agent run, the factory returns an AgentContext with new WorkingMemory, the scoped LongTermMemory, namespace, session ID, and principal. The service calls CoalaAgentDefinition.of to build the definition and ActionRegistry, then `new CoalaAgent(definition)` and `agent.run(new AgentRunRequest(...))`.

The definition makes the selected procedure, action allow-list, context factory, and max-cycle bound explicit. In this current demo wiring, trace listeners and completion listeners default to empty lists; no post-run hook is registered by this service path.

Actual source excerpt: dev/coala/demo/DemoUseCaseService.java (source lines 145-168)

```
    private AgentRunResult runAgent(String agentId, CoalaDecisionProcedure procedure,
                                    List<? extends CoalaAction> actions, String goal, int maxCycles,
                                    SessionMemoryStore.Session session) {
        return runAgent(agentId, procedure, actions, goal, maxCycles, session, MemoryNamespace.DEFAULT);
    }

    private AgentRunResult runAgent(String agentId, CoalaDecisionProcedure procedure,
                                    List<? extends CoalaAction> actions, String goal, int maxCycles,
                                    SessionMemoryStore.Session session, MemoryNamespace namespace) {
        AgentContextFactory contextFactory = AgentContextFactory.scoped(request -> session.longTermMemory());
        CoalaAgentDefinition definition = CoalaAgentDefinition.of(agentId, procedure, actions, contextFactory,
        maxCycles);
        CoalaAgent agent = new CoalaAgent(definition);
        return agent.run(new AgentRunRequest(goal, session.sessionId(), "demo", namespace));
    }

    private static MemoryNamespace namespace(String agentId, String sessionId, String domain) {
        return new MemoryNamespace(agentId, sessionId, "demo", domain);
    }

    private static void ensureSeed(SessionMemoryStore.Session session, MemoryNamespace namespace,
                                   String seedId, String content) {
        boolean exists = session.longTermMemory().list(namespace, 1_000).stream()
                .anyMatch(record -> seedId.equals(record.metadata().get("demoSeed")));
        if (!exists) {
```

Important boundary
The CoalaAgent facade is reusable, but this demo service currently constructs a new definition and agent facade for each request instead of registering six long-lived agent instances.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

AgentContextFactory creates one run-local state container

AgentContextFactory.scoped applies the resolver to AgentRunRequest and always creates `new WorkingMemory()`. This is why the current cycle state such as latest_observation, plan.candidates, travel.flight_options, or research.brief is not automatically carried into the next HTTP run. Only the supplied long-term memory object is shared across requests in a session.

The namespace, sessionId and principal are copied into AgentContext. These values travel with memory operations; the namespace is what makes the scoped retrieval/write boundaries meaningful. The current demo principal is the literal `demo`, which is not user authentication.

Actual source excerpt: dev/coala/core/agent/AgentContextFactory.java (source lines 1-25)

```
package dev.coala.core.agent;

import dev.coala.core.AgentContext;
import dev.coala.core.WorkingMemory;
import dev.coala.core.memory.LongTermMemory;

import java.util.Objects;
import java.util.function.Function;
import java.util.function.Supplier;

/** Creates isolated per-run working memory and supplies appropriately scoped long-term memory. */
@FunctionalInterface
public interface AgentContextFactory {
    AgentContext create(AgentRunRequest request);

    static AgentContextFactory isolated(Supplier<? extends LongTermMemory> memorySupplier) {
        Objects.requireNonNull(memorySupplier, "memorySupplier");
        return request -> new AgentContext(new WorkingMemory(), Objects.requireNonNull(memorySupplier.get(),
        "memory supplier returned null"), request.namespace(), request.sessionId(), request.principal());
    }

    static AgentContextFactory scoped(Function<AgentRunRequest, ? extends LongTermMemory> memoryResolver) {
        Objects.requireNonNull(memoryResolver, "memoryResolver");
        return request -> new AgentContext(new WorkingMemory(),
        Objects.requireNonNull(memoryResolver.apply(request), "memory resolver returned null"),
        request.namespace(), request.sessionId(), request.principal());
    }
}
```

Important boundary
A real multitenant app must resolve principal and tenant identity from authenticated application context, not from a caller-controlled string.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

CoalaAgent.run creates run identity and calls the procedure

CoalaAgent.run(AgentRunRequest) creates a random run UUID and calls definition.contextFactory().create(request). It publishes RUN_STARTED to trace listeners, invokes `definition.decisionProcedure().run(goal, context, actions, maxCycles)`, publishes each returned DecisionEvent, invokes completion listeners, publishes RUN_COMPLETED, and returns the AgentRunResult.

The trace listener list is empty in this use-case builder, so publish iterates zero listeners. The method still owns the instrumentation boundary. If a runtime exception escapes, it publishes a generic RUN_FAILED trace without exposing the exception message, then rethrows it for the API exception handler.

Method-call ledger

01	Create runId UUID.
02	Create run-local AgentContext using AgentContextFactory.
03	Publish RUN_STARTED (no-op with the current empty listener list).
04	Call the selected CoalaDecisionProcedure.run(...).
05	Publish returned decision events (also no-op with no listeners).
06	Run completion listeners if any; none are configured here.
07	Return AgentRunResult or rethrow a sanitized-trace failure.

Actual source excerpt: dev/coala/core/agent/CoalaAgent.java (source lines 18-48)

```
    public AgentRunResult run(String goal) { return run(AgentRunRequest.goal(goal)); }

    public AgentRunResult run(AgentRunRequest request) {
        UUID runId = UUID.randomUUID();
        AgentContext context = definition.contextFactory().create(request);
        if (context == null) throw new IllegalStateException("AgentContextFactory returned null");
        publish(new DecisionTrace(runId, id(), request.sessionId(), "RUN_STARTED", null, "", "Agent run
        started", null));
        try {
            AgentRunResult result = definition.decisionProcedure().run(request.goal(), context,
                    definition.actions(), definition.maxCycles());
            for (DecisionEvent event : result.events()) publish(DecisionTrace.from(runId, id(),
        request.sessionId(), event));
            for (AgentRunCompletionListener listener : definition.completionListeners()) {
                try { listener.onCompleted(result, context); }
                catch (RuntimeException ignored) { /* Learning hooks are opt-in and cannot rewrite the completed
         result. */ }
            }
            publish(new DecisionTrace(runId, id(), request.sessionId(), "RUN_COMPLETED",
        result.cyclesExecuted(),
                    result.terminationReason().name(), "Agent run completed", null));
            return result;
        } catch (RuntimeException ex) {
            // Do not leak exception messages, prompts, tokens, or action arguments to generic traces.
            publish(new DecisionTrace(runId, id(), request.sessionId(), "RUN_FAILED", context.cycle(),
                    ex.getClass().getSimpleName(), "Agent run failed; inspect controlled application logs",
        null));
            throw ex;
        }
    }

    private void publish(DecisionTrace trace) {
        for (DecisionTraceListener listener : definition.traceListeners()) {
            try { listener.onTrace(trace); }
            catch (RuntimeException ignored) { /* Observability must not change decision semantics. */ }
        }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The shared decision loop initializes working memory

AbstractLoopingDecisionProcedure.run is the shared execution skeleton. It rejects blank or oversized goals and maxCycles outside 1..100, trims the goal, then writes goal and the initial latest_observation string into WorkingMemory. The first loop iteration begins at cycle one; each cycle stores that number on AgentContext.

Before planning, it records an OBSERVATION event and clears transient plan-selection fields so a previous cycle's candidate or selected_action cannot masquerade as the current decision. It does not clear domain outputs such as web_search.results, research.brief, travel.calendar, or support.triage; those are intentionally retained within the same run.

Actual source excerpt: dev/coala/core/procedure/AbstractLoopingDecisionProcedure.java (source lines 25-60)

```
 */
public abstract class AbstractLoopingDecisionProcedure implements CoalaDecisionProcedure {
    public static final int MAX_GOAL_CHARACTERS = 4_000;

    @Override
    public final AgentRunResult run(String goal, AgentContext context, ActionRegistry actions, int maxCycles) {
        if (goal == null || goal.isBlank()) throw new IllegalArgumentException("goal must not be blank");
        if (goal.length() > MAX_GOAL_CHARACTERS) throw new IllegalArgumentException("goal must be no more than "
         + MAX_GOAL_CHARACTERS + " characters");
        goal = goal.trim();
        if (maxCycles < 1 || maxCycles > 100) throw new IllegalArgumentException("maxCycles must be between 1
        and 100");

        context.workingMemory().put("goal", goal.trim());
        context.workingMemory().put("latest_observation", "User request received: " + goal.trim());
        List<DecisionEvent> events = new ArrayList<>();
        String finalObservation = "";

        for (int cycle = 1; cycle <= maxCycles; cycle++) {
            context.setCycle(cycle);
            String observation = context.workingMemory().getString("latest_observation", "");
            events.add(new DecisionEvent(cycle, DecisionStage.OBSERVATION,
                    truncate(observation, 240)));

            // Do not let a previous cycle's plan be mistaken for the current selection.
            context.workingMemory().put("plan.candidates", null);
            context.workingMemory().put("plan.evaluations", null);
            context.workingMemory().put("plan.selectedAction", null);
            context.workingMemory().put("plan.selectionRejected", null);
            context.workingMemory().put("plan.last_rejection", null);
            context.workingMemory().put("plan.previousRejectedCandidates", null);
            context.workingMemory().put("selected_action", null);
            context.workingMemory().put("selected_action_id", null);
            context.workingMemory().put("selected_action_arguments", null);

            PlanDecision plan;
            try {
                plan = planNextAction(context, actions);
```

Important boundary
The observation is the action result from the preceding cycle, not an automatic dump of all working memory.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

ActionRegistry is an allow-list, not an LLM tool executor

ActionRegistry’s constructor rejects null entries, blank IDs, descriptor/action ID mismatches, duplicates, and empty registries. `require(id)` resolves a string only from this registry. `validate(call)` calls require and then the registered action’s validate method, which enforces required/optional argument definitions, allowed fields, value type/length/range constraints, and contract consistency.

The model returns an ActionCall, which is data. No generated shell command, Java, script, or arbitrary reflection target is executed. The selected registered object is the one whose execute method the base loop calls.

Actual source excerpt: dev/coala/core/ActionRegistry.java (source lines 13-48)

```
    public ActionRegistry(Collection<? extends CoalaAction> actionList) {
        Objects.requireNonNull(actionList, "actionList");
        Map<String, CoalaAction> found = new LinkedHashMap<>();
        for (CoalaAction action : actionList) {
            if (action == null) throw new IllegalArgumentException("Action list contains null");
            if (action.id() == null || action.id().isBlank()) throw new IllegalArgumentException("Action id must
             not be blank");
            ActionDescriptor descriptor = action.descriptor();
            if (!action.id().equals(descriptor.id())) {
                throw new IllegalArgumentException("Action descriptor id does not match action id: " +
        action.id());
            }
            if (found.putIfAbsent(action.id(), action) != null) {
                throw new IllegalArgumentException("Duplicate action id: " + action.id());
            }
        }
        if (found.isEmpty()) throw new IllegalArgumentException("At least one action is required");
        this.actions = Collections.unmodifiableMap(new LinkedHashMap<>(found));
    }

    public CoalaAction require(String id) {
        CoalaAction action = actions.get(id);
        if (action == null) throw new DecisionProcedureException("Unknown action id proposed by the model: " +
        id);
        return action;
    }

    /** Resolve the action against the allow-list and enforce its declared argument contract. */
    public CoalaAction validate(ActionCall call) {
        CoalaAction action = require(call.actionId());
        action.validate(call);
        return action;
    }

    public List<ActionDescriptor> descriptors() {
        return actions.values().stream().map(CoalaAction::descriptor).toList();
    }
```

Important boundary
For higher-risk production actions, input validation must be followed by authorization and approval policy checks at the action boundary.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

ActionCall and ActionOutputParser make the model output structured

ActionCall is an immutable record: actionId plus string-valued arguments. Its constructor rejects missing IDs, excessive argument counts, oversized keys/values, and excessive aggregate argument text. The parser uses Jackson 3 strict features to reject duplicate JSON object keys and trailing JSON values.

Each procedure chooses its parser method: parseAction for one action, parseCandidates for at least two distinct alternatives, parseEvaluation for one finite score per candidate plus rejectAll, or parseActionPlan for a bounded list of steps. Only the shape chosen by the procedure is accepted; the parser never executes model output.

Actual source excerpt: dev/coala/core/action/ActionOutputParser.java (source lines 20-71)

```
public final class ActionOutputParser {
    private final ObjectMapper mapper;

    public ActionOutputParser() {
        // Jackson 3 is Spring Boot 4's default. Reject duplicate object keys and
        // trailing JSON values so ambiguous model output fails closed.
        this.mapper = JsonMapper.builder()
                .enable(DeserializationFeature.FAIL_ON_READING_DUP_TREE_KEY)
                .enable(DeserializationFeature.FAIL_ON_TRAILING_TOKENS)
                .build();
    }

    public ActionCall parseAction(String raw) {
        JsonNode root = parseJson(raw, "action proposal");
        requireObject(root, "Action JSON must be an object.");
        requireOnlyFields(root, Set.of("actionId", "arguments"), "action proposal");
        JsonNode actionId = root.get("actionId");
        if (actionId == null || !actionId.isTextual() || actionId.asText().isBlank()) {
            throw new InvalidModelOutputException("Action JSON must contain a non-empty string 'actionId'.");
        }
        return buildActionCall(actionId.asText(), parseArguments(root.get("arguments")));
    }

    /** Parses a set of at least two distinct alternatives. */
    public List<ActionCall> parseCandidates(String raw, int maxCandidates) {
        if (maxCandidates < 2) throw new IllegalArgumentException("maxCandidates must be at least 2");
        JsonNode root = parseJson(raw, "candidate proposal");
        requireObject(root, "Candidate JSON must be an object.");
        requireOnlyFields(root, Set.of("candidates"), "candidate proposal");
        JsonNode candidates = root.get("candidates");
        if (candidates == null || !candidates.isArray() || candidates.size() < 2) {
            throw new InvalidModelOutputException("Candidate JSON must contain at least two alternatives in a
        'candidates' array.");
        }
        if (candidates.size() > maxCandidates) {
            throw new InvalidModelOutputException("Model proposed " + candidates.size()
                    + " candidates; configured limit is " + maxCandidates + ".");
        }
        List<ActionCall> parsed = new ArrayList<>();
        Set<ActionCall> seen = new HashSet<>();
        for (JsonNode candidate : candidates) {
            requireObject(candidate, "Every candidate must be a JSON object.");
            requireOnlyFields(candidate, Set.of("actionId", "arguments"), "candidate action");
            JsonNode id = candidate.get("actionId");
            if (id == null || !id.isTextual() || id.asText().isBlank()) {
                throw new InvalidModelOutputException("Every candidate must contain a non-empty 'actionId'.");
            }
            ActionCall call = buildActionCall(id.asText(), parseArguments(candidate.get("arguments")));
            if (!seen.add(call)) {
                throw new InvalidModelOutputException("Candidate set contains duplicate action calls; propose
        distinct alternatives.");
            }
            parsed.add(call);
        }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

WorkingMemory.put/get/snapshot are state-changing operations, not passive maps

WorkingMemory is the active symbolic state for one run. put(key, value) first rejects null/blank keys and calls freeze(value) before taking the write lock. freeze accepts a small set of scalar types and recursively copies Maps, Lists and Sets into unmodifiable collections. This prevents a caller from mutating a nested map after it has been stored. Unsupported objects, non-string map keys, and null items inside collections fail fast rather than silently entering prompt state.

A top-level null has a different meaning from a null nested value: put(key, null) removes the key. The procedure relies on that behavior to clear stale plan.candidates, selected_action, and reasoning.latest. get(key) returns an Optional under the read lock; getString maps a present value through Object.toString or uses a fallback; snapshot returns an unmodifiable copy of the current top-level map. renderForPrompt renders a detached snapshot after releasing the lock, so prompt formatting does not hold the lock while doing string work.

For each cycle, the state must be read as a sequence of snapshots and writes. For example, after a search action writes web_search.results, cycle 2 does not reconstruct those results from the prior observation; it sees the domain key preserved in the same WorkingMemory object. A new HTTP request creates a new WorkingMemory, even when its sessionId resumes the same long-term memory.

Method-call ledger

01	WorkingMemory.put(key, value) checks the key.
02	freeze(value) defensively copies and freezes supported nested collections.
03	Acquire writeLock; if safeValue == null remove key, otherwise put key/value.
04	get(key) acquires readLock and returns Optional.ofNullable(values.get(key)).
05	getString(key, fallback) converts present values with Object.toString; absent keys use fallback.
06	snapshot() returns an unmodifiable copy made under readLock.
07	renderForPrompt calls snapshot, then formats outside the lock.

State and observable values

Field	Value / meaning
top-level null	means remove that key
nested null	rejected inside maps/lists/sets
cross-cycle	same WorkingMemory for all cycles in one run
cross-request	not retained; only long-term memory is shared by session

Actual source excerpt: dev/coala/core/WorkingMemory.java (source lines 20-105)

```
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();
    private final LinkedHashMap<String, Object> values = new LinkedHashMap<>();

    public void put(String key, Object value) {
        if (key == null || key.isBlank()) throw new IllegalArgumentException("key must not be blank");
        Object safeValue = value == null ? null : freeze(value);
        lock.writeLock().lock();
        try {
            if (safeValue == null) values.remove(key);
            else values.put(key, safeValue);
        } finally {
            lock.writeLock().unlock();
        }
    }

    /**
     * Working memory is symbolic state. Defensively freeze supported collection
     * types so snapshots/getters cannot be mutated through nested object references.
     */
    private Object freeze(Object value) {
        if (value instanceof String || value instanceof Boolean || value instanceof Byte
                || value instanceof Short || value instanceof Integer || value instanceof Long
                || value instanceof Float || value instanceof Double || value instanceof BigInteger
                || value instanceof BigDecimal || value instanceof Enum<?>) {
            return value;
        }
        if (value instanceof Map<?, ?> map) {
            Map<String, Object> frozen = new LinkedHashMap<>();
            map.forEach((key, nestedValue) -> {
                if (!(key instanceof String textKey) || textKey.isBlank()) {
                    throw new IllegalArgumentException("Working-memory map keys must be non-blank strings");
                }
                if (nestedValue == null) throw new IllegalArgumentException("Nested working-memory values must
        not be null");
                frozen.put(textKey, freeze(nestedValue));
            });
            return Collections.unmodifiableMap(frozen);
```

```
        }
        if (value instanceof List<?> list) {
            List<Object> frozen = new ArrayList<>(list.size());
            for (Object item : list) {
                if (item == null) throw new IllegalArgumentException("Nested working-memory values must not be
        null");
                frozen.add(freeze(item));
            }
            return Collections.unmodifiableList(frozen);
        }
        if (value instanceof Set<?> set) {
            Set<Object> frozen = new LinkedHashSet<>();
            for (Object item : set) {
                if (item == null) throw new IllegalArgumentException("Nested working-memory values must not be
        null");
                frozen.add(freeze(item));
            }
            return Collections.unmodifiableSet(frozen);
        }
        throw new IllegalArgumentException("Unsupported working-memory value type: " +
        value.getClass().getName());
    }

    public Optional<Object> get(String key) {
        lock.readLock().lock();
        try {
            return Optional.ofNullable(values.get(key));
        } finally {
            lock.readLock().unlock();
        }
    }

    public String getString(String key, String fallback) {
        return get(key).map(Object::toString).orElse(fallback);
    }

    public Map<String, Object> snapshot() {
        lock.readLock().lock();
        try {
            return Collections.unmodifiableMap(new LinkedHashMap<>(values));
        } finally {
            lock.readLock().unlock();
        }
    }

    public String renderForPrompt() {
        // Values are frozen on entry; format a detached snapshot without holding the lock.
        Map<String, Object> detached = snapshot();
        StringBuilder out = new StringBuilder();
        detached.forEach((key, value) -> out.append(key).append(" = ").append(value).append('\n'));
        return out.toString();
    }
}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Argument contracts are two layers: shape, then action-specific validation

ActionOutputParser first enforces the JSON/data shape. ActionCall then bounds the action ID, number of arguments, key length, value length, and aggregate argument text. ActionRegistry.validate resolves the ID only against the registered allow-list and calls CoalaAction.validate. The default CoalaAction contract builds ActionArgumentDefinition entries from requiredArguments and optionalArguments; a concrete action can override argumentDefinitions to declare INTEGER, DECIMAL, BOOLEAN, ENUM, or URI types and length limits.

ActionArgumentDefinition.validate checks one value: null is rejected; excessive length is rejected; required blank values are rejected; an optional blank value returns early; then the selected type parser runs. INTEGER uses Long.parseLong, DECIMAL uses BigDecimal with precision/scale bounds, BOOLEAN accepts only true/false, ENUM checks exact membership, and URI requires an absolute URI. CoalaAction.validate additionally rejects unknown argument names and missing required names before validating each supplied value.

These contracts validate syntax and allowed fields; they do not prove a value is semantically appropriate. For example, an integer maxResults can parse successfully but still exceed a provider cap, so WebSearchGroundingAction applies its own domain check. Likewise, travel search actions separately require the calendar.checked precondition. Validation must not be mistaken for authorization or approval.

Method-call ledger

01	ActionRegistry.validate(ActionCall) calls require(actionId).
02	require finds the registered CoalaAction or rejects an unknown ID.
03	CoalaAction.validate checks the call is non-null and builds the action’s definitions.
04	Build allowed and required name sets; reject duplicate/null definitions.
05	Reject unknown argument keys and missing required keys.
06	For each supplied argument, ActionArgumentDefinition.validate checks length and required blankness.
07	Switch on ValueType to parse and validate the typed string.
08	Concrete execute method performs domain preconditions after contract validation.

Actual source excerpt: dev/coala/core/ActionArgumentDefinition.java (source lines 1-66)

```
package dev.coala.core;

import java.math.BigDecimal;
import java.net.URI;
import java.util.LinkedHashSet;
import java.util.Set;

/** Typed input contract for a registered action. This is an implementation-level validation aid. */
public record ActionArgumentDefinition(
        String name,
        ValueType type,
        String description,
        boolean required,
        Set<String> allowedValues,
        int maxLength) {

    public enum ValueType { STRING, INTEGER, DECIMAL, BOOLEAN, ENUM, URI }

    public ActionArgumentDefinition {
        if (name == null || name.isBlank() || name.length() > ActionCall.MAX_ARGUMENT_KEY_LENGTH)
            throw new IllegalArgumentException("Argument name must be non-blank and at most " +
        ActionCall.MAX_ARGUMENT_KEY_LENGTH + " characters");
        if (type == null) throw new IllegalArgumentException("Argument type is required");
        description = description == null ? "" : description;
        allowedValues = allowedValues == null ? Set.of() : Set.copyOf(new LinkedHashSet<>(allowedValues));
        if (maxLength < 1 || maxLength > ActionCall.MAX_ARGUMENT_VALUE_LENGTH)
            throw new IllegalArgumentException("maxLength must be between 1 and " +
        ActionCall.MAX_ARGUMENT_VALUE_LENGTH);
        if (type == ValueType.ENUM && allowedValues.isEmpty()) throw new IllegalArgumentException("ENUM
        arguments must declare allowedValues");
        if (type != ValueType.ENUM && !allowedValues.isEmpty()) throw new IllegalArgumentException("Only ENUM
        arguments may declare allowedValues");
    }

    public static ActionArgumentDefinition requiredString(String name, String description) {
        return new ActionArgumentDefinition(name, ValueType.STRING, description, true, Set.of(), 4_000);
    }
    public static ActionArgumentDefinition optionalString(String name, String description) {
        return new ActionArgumentDefinition(name, ValueType.STRING, description, false, Set.of(), 4_000);
    }

    public void validate(String value) {
        if (value == null) throw new DecisionProcedureException("Argument '" + name + "' must not be null.");
        if (value.length() > maxLength) throw new DecisionProcedureException("Argument '" + name + "' exceeds
        its maximum length of " + maxLength + ".");
        if (required && value.isBlank()) throw new DecisionProcedureException("Required argument '" + name + "'
        must not be blank.");
        if (value.isBlank() && !required) return;
```

```
        try {
            switch (type) {
                case STRING -> { }
                case INTEGER -> Long.parseLong(value.trim());
                case DECIMAL -> {
                    BigDecimal decimal = new BigDecimal(value.trim());
                    if (decimal.precision() > 100 || Math.abs(decimal.scale()) > 100) throw new
        NumberFormatException("numeric value outside supported precision");
                }
                case BOOLEAN -> {
                    if (!value.equalsIgnoreCase("true") && !value.equalsIgnoreCase("false")) throw new
        IllegalArgumentException("must be true or false");
                }
                case ENUM -> {
                    if (!allowedValues.contains(value)) throw new IllegalArgumentException("must be one of " +
        allowedValues);
                }
                case URI -> {
                    URI uri = URI.create(value.trim());
                    if (!uri.isAbsolute()) throw new IllegalArgumentException("must be an absolute URI");
                }
            }
        } catch (RuntimeException ex) {
            throw new DecisionProcedureException("Argument '" + name + "' is not a valid " + type + " value: " +
         ex.getMessage(), ex);
        }
    }
}
```

Important boundary
The actual interface validation continues in CoalaAction.java; argument validation is one part of an action boundary, not a substitute for safety rules inside the action.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Strict JSON parsing has its own nested call path

ActionOutputParser is not a single “parse JSON” call. parseAction, parseCandidates, parseEvaluation and parseActionPlan each invoke parseJson first, then requireObject, requireOnlyFields, and their specific structure checks. They call parseArguments to force every model-supplied argument to be a JSON string, then buildActionCall. buildActionCall delegates final size/shape constraints to the ActionCall constructor and converts an IllegalArgumentException into InvalidModelOutputException.

parseJson rejects null/blank strings and outputs over 64,000 characters. It optionally removes exactly one outer fenced JSON block only when the closing fence is well formed and no trailing content remains. Jackson’s configured strict mapper then reads the JSON tree; malformed JSON, duplicate keys, or trailing JSON values become InvalidModelOutputException. This exception is caught by the API advice and returned as a sanitized 502 for invalid model output.

Different procedures use different parsers. Iterative reasoning uses parseAction for one action object. Deliberative planning uses parseCandidates, which requires at least two distinct ActionCall values, then parseEvaluation, which requires one finite numeric score per candidate plus a boolean rejectAll. Plan-first uses parseActionPlan, which enforces a non-empty bounded steps array. Parsing never resolves the action implementation and never executes anything; ActionRegistry performs the next validation step.

Method-call ledger

01	Procedure receives raw string from ReasoningAction.reason.
02	Call the procedure-specific parser: parseAction / parseCandidates / parseEvaluation / parseActionPlan.
03	parseJson checks empty/size/fenced form and invokes Jackson readTree.
04	requireObject rejects scalar/array roots where an object is required.
05	requireOnlyFields rejects unknown JSON properties.
06	Procedure-specific code validates arrays, booleans, scores, IDs, or step counts.
07	parseArguments converts only textual JSON values into Map<String,String>.
08	buildActionCall invokes ActionCall constructor and translates contract errors.
09	Procedure/loop next passes ActionCall to ActionRegistry.validate; only then can execution happen.

Actual source excerpt: dev/coala/core/action/ActionOutputParser.java (source lines 20-205)

```
public final class ActionOutputParser {
    private final ObjectMapper mapper;

    public ActionOutputParser() {
        // Jackson 3 is Spring Boot 4's default. Reject duplicate object keys and
        // trailing JSON values so ambiguous model output fails closed.
        this.mapper = JsonMapper.builder()
                .enable(DeserializationFeature.FAIL_ON_READING_DUP_TREE_KEY)
                .enable(DeserializationFeature.FAIL_ON_TRAILING_TOKENS)
                .build();
    }

    public ActionCall parseAction(String raw) {
        JsonNode root = parseJson(raw, "action proposal");
        requireObject(root, "Action JSON must be an object.");
        requireOnlyFields(root, Set.of("actionId", "arguments"), "action proposal");
        JsonNode actionId = root.get("actionId");
        if (actionId == null || !actionId.isTextual() || actionId.asText().isBlank()) {
            throw new InvalidModelOutputException("Action JSON must contain a non-empty string 'actionId'.");
        }
        return buildActionCall(actionId.asText(), parseArguments(root.get("arguments")));
    }

    /** Parses a set of at least two distinct alternatives. */
    public List<ActionCall> parseCandidates(String raw, int maxCandidates) {
        if (maxCandidates < 2) throw new IllegalArgumentException("maxCandidates must be at least 2");
        JsonNode root = parseJson(raw, "candidate proposal");
        requireObject(root, "Candidate JSON must be an object.");
        requireOnlyFields(root, Set.of("candidates"), "candidate proposal");
        JsonNode candidates = root.get("candidates");
        if (candidates == null || !candidates.isArray() || candidates.size() < 2) {
            throw new InvalidModelOutputException("Candidate JSON must contain at least two alternatives in a
        'candidates' array.");
        }
        if (candidates.size() > maxCandidates) {
            throw new InvalidModelOutputException("Model proposed " + candidates.size()
                    + " candidates; configured limit is " + maxCandidates + ".");
        }
        List<ActionCall> parsed = new ArrayList<>();
        Set<ActionCall> seen = new HashSet<>();
        for (JsonNode candidate : candidates) {
            requireObject(candidate, "Every candidate must be a JSON object.");
            requireOnlyFields(candidate, Set.of("actionId", "arguments"), "candidate action");
            JsonNode id = candidate.get("actionId");
            if (id == null || !id.isTextual() || id.asText().isBlank()) {
                throw new InvalidModelOutputException("Every candidate must contain a non-empty 'actionId'.");
            }
```

```
        ActionCall call = buildActionCall(id.asText(), parseArguments(candidate.get("arguments")));
        if (!seen.add(call)) {
            throw new InvalidModelOutputException("Candidate set contains duplicate action calls; propose
    distinct alternatives.");
        }
        parsed.add(call);
    }
    return List.copyOf(parsed);
}

/**
 * Parses an explicit selection decision and one score per candidate. rejectAll=true
 * means no candidate is selected and planning returns to proposal.
 */
public CandidateEvaluation parseEvaluation(String raw, int expectedCandidates) {
    if (expectedCandidates < 2) throw new IllegalArgumentException("expectedCandidates must be at least 2");
    JsonNode root = parseJson(raw, "candidate evaluation");
    requireObject(root, "Evaluation JSON must be an object.");
    requireOnlyFields(root, Set.of("rejectAll", "scores"), "candidate evaluation");
    JsonNode reject = root.get("rejectAll");
    if (reject == null || !reject.isBoolean()) {
        throw new InvalidModelOutputException("Evaluation JSON must contain boolean 'rejectAll'.");
    }
    JsonNode scoreNode = root.get("scores");
    if (scoreNode == null || !scoreNode.isArray() || scoreNode.isEmpty()) {
        throw new InvalidModelOutputException("Evaluation JSON must contain a non-empty 'scores' array.");
    }
    if (scoreNode.size() != expectedCandidates) {
        throw new InvalidModelOutputException("Evaluator returned " + scoreNode.size() + " score(s) for " +
    expectedCandidates + " candidate(s).");
    }
    List<Double> scores = new ArrayList<>();
    for (JsonNode score : scoreNode) {
        if (!score.isNumber()) throw new InvalidModelOutputException("Every candidate score must be
    numeric.");
        double value = score.asDouble();
        if (!Double.isFinite(value)) throw new InvalidModelOutputException("Candidate scores must be finite
    numbers.");
        scores.add(value);
    }
    return new CandidateEvaluation(reject.asBoolean(), scores);
}


/** Parses a bounded structured plan of executable action calls. */
public List<ActionCall> parseActionPlan(String raw, int maxSteps) {
    if (maxSteps < 1 || maxSteps > 20) throw new IllegalArgumentException("maxSteps must be 1..20");
    JsonNode root = parseJson(raw, "action plan");
    requireObject(root, "Plan JSON must be an object.");
    requireOnlyFields(root, Set.of("steps"), "action plan");
    JsonNode steps = root.get("steps");
    if (steps == null || !steps.isArray() || steps.isEmpty() || steps.size() > maxSteps)
        throw new InvalidModelOutputException("Plan must contain between 1 and " + maxSteps + " steps.");
    List<ActionCall> parsed = new ArrayList<>();
    for (JsonNode candidate : steps) {
        requireObject(candidate, "Every plan step must be an object.");
        requireOnlyFields(candidate, Set.of("actionId", "arguments"), "plan step");
        JsonNode id = candidate.get("actionId");
        if (id == null || !id.isTextual() || id.asText().isBlank()) throw new
    InvalidModelOutputException("Each plan step requires a non-empty actionId.");
        parsed.add(buildActionCall(id.asText(), parseArguments(candidate.get("arguments"))));
    }
    return List.copyOf(parsed);
}

public ActionQualityEvaluation parseQualityEvaluation(String raw) {
    JsonNode root = parseJson(raw, "action quality evaluation");
    requireObject(root, "Quality evaluation JSON must be an object.");
    requireOnlyFields(root, Set.of("accepted", "score", "feedback"), "quality evaluation");
    JsonNode accepted = root.get("accepted"), score = root.get("score"), feedback = root.get("feedback");
    if (accepted == null || !accepted.isBoolean() || score == null || !score.isNumber() || feedback == null
    || !feedback.isTextual())
        throw new InvalidModelOutputException("Quality evaluation requires boolean accepted, numeric score
    and string feedback.");
    double value = score.asDouble();
    if (!Double.isFinite(value) || value < 0 || value > 1) throw new InvalidModelOutputException("Quality
    score must be finite and between 0 and 1.");
    String note = feedback.asText();
    if (note.length() > 1_000) note = note.substring(0, 997) + "...";
    return new ActionQualityEvaluation(accepted.asBoolean(), value, note);
}

/** Retained as a utility for tests and callers that only need score parsing. */
public List<Double> parseScores(String raw) {
    JsonNode root = parseJson(raw, "candidate evaluation");
    requireObject(root, "Evaluation JSON must be an object.");
    requireOnlyFields(root, Set.of("scores"), "candidate score result");
    JsonNode scores = root.get("scores");
    if (scores == null || !scores.isArray() || scores.isEmpty()) {
        throw new InvalidModelOutputException("Evaluation JSON must contain a non-empty 'scores' array.");
    }
    List<Double> result = new ArrayList<>();
    for (JsonNode score : scores) {
        if (!score.isNumber() || !Double.isFinite(score.asDouble())) {
            throw new InvalidModelOutputException("Every candidate score must be a finite number.");
        }
        result.add(score.asDouble());
    }
    return List.copyOf(result);
}
```

```
private Map<String, String> parseArguments(JsonNode node) {
    if (node == null || node.isNull() || !node.isObject()) {
        throw new InvalidModelOutputException("'arguments' must be a JSON object.");
    }
    if (node.size() > ActionCall.MAX_ARGUMENTS) {
        throw new InvalidModelOutputException("Action JSON contains too many arguments.");
    }
    Map<String, String> args = new LinkedHashMap<>();
    node.fields().forEachRemaining(entry -> {
        JsonNode value = entry.getValue();
        if (!value.isTextual()) {
            throw new InvalidModelOutputException("Action argument '" + entry.getKey() + "' must be a
    string.");
        }
        args.put(entry.getKey(), value.asText());
    });
    return Map.copyOf(args);
}


private ActionCall buildActionCall(String actionId, Map<String, String> arguments) {
    try {
        return new ActionCall(actionId, arguments);
    } catch (IllegalArgumentException ex) {
        throw new InvalidModelOutputException("Action proposal violates the action-call data contract: " +
    ex.getMessage(), ex);
    }
}

private void requireOnlyFields(JsonNode node, Set<String> allowed, String description) {
    var fields = node.fields();
    while (fields.hasNext()) {
        String field = fields.next().getKey();
        if (!allowed.contains(field)) {
            throw new InvalidModelOutputException("Unexpected field '" + field + "' in " + description +
    ".");
        }
    }
}

private void requireObject(JsonNode node, String message) {
    if (node == null || !node.isObject()) throw new InvalidModelOutputException(message);
}

private JsonNode parseJson(String raw, String description) {
    if (raw == null || raw.isBlank()) throw new InvalidModelOutputException("Empty " + description + "
    output.");
    if (raw.length() > 64_000) throw new InvalidModelOutputException(description + " exceeds the maximum
    allowed size.");
    String text = raw.trim();
    if (text.startsWith("```")) {
        int firstNewline = text.indexOf('\n');
```

Important boundary
A syntactically valid action JSON can still name an unregistered action or fail an action’s value contract. The parser, registry and concrete action each protect a different boundary.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

ReasoningAction.reason assembles a bounded, sanitized prompt

LlmReasoningAction increments a per-run reasoning.call_count and enforces its configured maximum. It asks PromptContextPolicy for keys that can be shown to the current operation, builds a bounded WorkingMemoryPromptContext, sanitizes the goal and full prompt using configured redaction patterns, and sends only the final prompt to LanguageModel.generate.

It rejects empty or excessively large model output, then stores reasoning.last_operation and reasoning.latest. The procedure-specific caller parses that string into an ActionCall, candidate list, evaluation, or plan, and clears reasoning.latest on the successful path so raw JSON does not remain as long-term state.

Method-call ledger

01	Read reasoning.call_count from WorkingMemory and enforce budget.
02	Select operation-specific prompt context with PromptContextPolicy.requestFor.
03	Build and bound a view using WorkingMemoryPromptContextBuilder.build.
04	Sanitize goal and complete assembled prompt.
05	Call LanguageModel.generate(prompt).
06	Reject empty/oversized output; store operation and raw output temporarily.

Actual source excerpt: dev/coala/core/action/LlmReasoningAction.java (source lines 35-79)

```
        }
        this.maxCallsPerRun = maxCallsPerRun;
        this.promptContextBuilder = Objects.requireNonNull(promptContextBuilder, "promptContextBuilder");
        this.promptContextPolicy = Objects.requireNonNull(promptContextPolicy, "promptContextPolicy");
    }

    @Override
    public String reason(String operation, String instruction, AgentContext context) {
        int callCount = Integer.parseInt(context.workingMemory().getString("reasoning.call_count", "0"));
        if (callCount >= maxCallsPerRun) {
            throw new DecisionProcedureException("Maximum reasoning-call budget exhausted for this run (" +
        maxCallsPerRun + ").");
        }
        context.workingMemory().put("reasoning.call_count", callCount + 1);
        WorkingMemoryPromptContextBuilder.Request request = promptContextPolicy.requestFor(operation, context);
        if (request == null) throw new IllegalStateException("PromptContextPolicy returned null for " +
        operation);
        WorkingMemoryPromptContextBuilder.PromptContext memoryView =
        promptContextBuilder.build(context.workingMemory(), request);
        String prompt = "You are performing an internal CoALA reasoning action.\n"
                + "OPERATION=" + operation + "\n"
                + "CYCLE=" + context.cycle() + "\n"
                + "GOAL=" + promptContextBuilder.sanitizeText(context.workingMemory().getString("goal", ""),
        request) + "\n"
                + "WORKING_MEMORY_BEGIN\n" + memoryView.rendered()
                + "WORKING_MEMORY_END\n"
                + instruction + "\n"
                + "Return only the requested result. Do not perform an external action yourself.";
        String generated;
        try {
            generated = languageModel.generate(promptContextBuilder.sanitizeText(prompt, request));
        } catch (LanguageModelException ex) {
            throw ex;
        } catch (RuntimeException ex) {
            throw new LanguageModelException("Language-model provider call failed for operation " + operation +
        ".", ex);
        }
        if (generated == null || generated.isBlank()) {
            throw new LanguageModelException("Language-model provider returned an empty response for operation "
         + operation + ".");
        }
        String normalized = generated.trim();
        if (normalized.length() > MAX_MODEL_OUTPUT_CHARACTERS) {
            throw new InvalidModelOutputException("Language-model output exceeds the maximum allowed size of "
                    + MAX_MODEL_OUTPUT_CHARACTERS + " characters.");
        }
        context.workingMemory().put("reasoning.last_operation", operation);
        context.workingMemory().put("reasoning.latest", normalized);
        return normalized;
    }
}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

PromptContextPolicy and WorkingMemoryPromptContextBuilder restrict context

The default prompt policy is an allow-list of named working-memory keys such as goal, latest_observation, retrieved_memories, plan candidates/evaluations, and domain-specific result keys. It can add approved namespaces such as web_search.*, research.*, travel.*, and support.*. It does not serialize every key automatically.

WorkingMemoryPromptContextBuilder snapshots memory, filters to allowed keys, rejects explicitly denied or credential-like key names, renders values, applies secret redaction patterns, and truncates at a bounded size. These steps reduce accidental prompt disclosure but do not make arbitrary user content trustworthy or replace data classification.

Actual source excerpt: dev/coala/core/PromptContextPolicy.java (source lines 12-33)

```
            keys.add("goal", "latest_observation", "latest_action", "retrieved_memories", "plan.candidates",
                    "plan.evaluations", "plan.last_rejection", "plan.previousRejectedCandidates",
        "plan.first_steps",
                    "plan.refinementCandidate", "plan.refinementEvaluation", "reasoning.call_count",
                    "reasoning.last_operation", "reflection.latest", "interaction.pending",
        "interaction.answer",
                    "retrieved_skills", "web_search.results", "calculator.lastResult",
        "calculator.lastExpression",
                    "research.topic", "research.brief", "research.source_count", "research.final",
                    "travel.calendar", "travel.destination", "travel.flight_options", "travel.hotel_options",
                    "travel.proposed_itinerary", "travel.estimate_cad", "travel.calendar.checked",
                    "travel.budget_cad", "travel.within_budget", "support.triage");
                // Built-in grounding actions use a per-action namespace for outputs. Include those
                // non-sensitive result keys dynamically without dumping unrelated working memory.
                context.workingMemory().snapshot().keySet().stream()
                        .filter(key -> key.startsWith("http.") || key.startsWith("workspace.") ||
        key.startsWith("web_search."))
                        .forEach(key -> keys.add(key));
                return new WorkingMemoryPromptContextBuilder.Request(keys.values(), java.util.Set.of(),
                        WorkingMemoryPromptContextBuilder.defaultRedactionPatterns(), 12_000, true);
            };
        }

        final class SetBuilder {
            private final java.util.Set<String> values = new java.util.LinkedHashSet<>();
            void add(String... names) { java.util.Collections.addAll(values, names); }
```

Important boundary
Model prompts contain user goals and tool observations. Treat both as untrusted input and keep action authorization in code rather than in prompt instructions.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

ScriptedLanguageModel selects deterministic fixtures

ScriptedLanguageModel is not an LLM. It parses the OPERATION marker and CYCLE integer from the prompt, then switches to a fixed fixture function. In the research path it chooses candidate JSON or scores based on whether the prompt contains `format_evidence_brief`; in travel it creates a fixed plan; in support it checks cycle and simple keywords in the goal.

This is a design choice for offline repeatability. It makes the examples good for teaching call sequencing and parsing, but it does not prove that a real model will always choose these actions, score these candidates similarly, or extract destinations and budgets correctly.

Actual source excerpt: dev/coala/llm/ScriptedLanguageModel.java (source lines 22-47)

```
        return switch (operation) {
            case "DIRECT_ACTION" -> "{\"actionId\":\"respond\",\"arguments\":{\"text\":\"This is the
        direct-action demo response. The request was handled by one selected dialogue grounding action; enable
        the openai profile to use an actual LLM.\"}}";
            case "PLAN_FIRST" -> planFirst(prompt);
            case "ITERATIVE_NEXT_ACTION" -> iterativeAction(cycle, prompt);
            case "PROPOSE_CANDIDATES" -> prompt.contains("format_evidence_brief") ? researchCandidates(cycle,
        prompt) : puzzleCandidates(cycle);
            case "EVALUATE_CANDIDATES" -> prompt.contains("format_evidence_brief") ? researchScores(cycle) :
        puzzleScores(cycle);
            default -> throw new IllegalArgumentException("Scripted adapter has no fixture for operation=" +
        operation);
        };
    }

    private String extractOperation(String prompt) {
        Matcher matcher = Pattern.compile("OPERATION=([A-Z_]+)").matcher(prompt);
        if (!matcher.find()) throw new IllegalArgumentException("Prompt has no OPERATION marker.");
        return matcher.group(1);
    }

    private int extractCycle(String prompt) {
        Matcher matcher = CYCLE.matcher(prompt);
        return matcher.find() ? Integer.parseInt(matcher.group(1)) : 1;
    }

    private String extractGoal(String prompt) {
        Matcher matcher = GOAL.matcher(prompt);
        return matcher.find() ? matcher.group(1).trim() : "the requested topic";
    }
```

Important boundary
The OpenAI profile changes the provider adapter; it does not remove the need for the same parser, action registry, state checks, limits and tests.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

What the offline language-model adapter actually calls

LanguageModel.generate(prompt) is the seam used by the cognitive framework. In the offline profile, the object behind that interface is ScriptedLanguageModel. Its generate method does not invoke a remote model: it calls extractOperation(prompt), extractCycle(prompt), and extractGoal(prompt), then routes the operation marker through a switch to the matching fixture method.

For this research run the operation marker is PROPOSE_CANDIDATES or EVALUATE_CANDIDATES and the cycle number chooses the fixed candidate or score JSON. For travel, operation PLAN_FIRST routes to planFirst(prompt), which calls extractDestination and extractBudget and constructs a six-step JSON plan. For support, ITERATIVE_NEXT_ACTION routes through iterativeAction to supportAction; the latter branches on cycle number and simple keywords in extractGoal(prompt).

This page is critical when interpreting the trace: a method named generate does not imply an LLM request. Spring AI is not contacted in this default profile. If the OpenAI profile is enabled, the provider adapter implements LanguageModel with a remote model call; the surrounding prompt assembly, parsing, action registry, procedure and execution path remain the same, but the exact returned JSON may differ.

Method-call ledger

01	LlmReasoningAction.reason calls languageModel.generate(sanitizedPrompt).
02	ScriptedLanguageModel.generate extracts operation marker.
03	extractCycle reads CYCLE=<integer>; extractGoal reads the GOAL line.
04	switch routes to planFirst, iterativeAction, researchCandidates, researchScores, or another fixture method.
05	The chosen fixture returns a JSON string; it has not called any action.
06	The procedure-specific parser validates that string before an ActionCall can be selected.

State and observable values

Field	Value / meaning
Research	operation + cycle determine the canned candidates/scores
Travel	PLAN_FIRST + regex-extracted destination/budget determine fixed plan JSON
Support	cycle 1 returns classification; cycle 2 uses goal keywords to return escalation
Provider I/O	none in scripted profile

Actual source excerpt: dev/coala/llm/ScriptedLanguageModel.java (source lines 19-58)

```
    public String generate(String prompt) {
        String operation = extractOperation(prompt);
        int cycle = extractCycle(prompt);
        return switch (operation) {
            case "DIRECT_ACTION" -> "{\"actionId\":\"respond\",\"arguments\":{\"text\":\"This is the
        direct-action demo response. The request was handled by one selected dialogue grounding action; enable
        the openai profile to use an actual LLM.\"}}";
            case "PLAN_FIRST" -> planFirst(prompt);
            case "ITERATIVE_NEXT_ACTION" -> iterativeAction(cycle, prompt);
            case "PROPOSE_CANDIDATES" -> prompt.contains("format_evidence_brief") ? researchCandidates(cycle,
        prompt) : puzzleCandidates(cycle);
            case "EVALUATE_CANDIDATES" -> prompt.contains("format_evidence_brief") ? researchScores(cycle) :
        puzzleScores(cycle);
            default -> throw new IllegalArgumentException("Scripted adapter has no fixture for operation=" +
        operation);
        };
    }

    private String extractOperation(String prompt) {
        Matcher matcher = Pattern.compile("OPERATION=([A-Z_]+)").matcher(prompt);
        if (!matcher.find()) throw new IllegalArgumentException("Prompt has no OPERATION marker.");
        return matcher.group(1);
    }

    private int extractCycle(String prompt) {
        Matcher matcher = CYCLE.matcher(prompt);
        return matcher.find() ? Integer.parseInt(matcher.group(1)) : 1;
    }

    private String extractGoal(String prompt) {
        Matcher matcher = GOAL.matcher(prompt);
        return matcher.find() ? matcher.group(1).trim() : "the requested topic";
    }

    private String extractDestination(String prompt) {
        Matcher matcher = DESTINATION.matcher(extractGoal(prompt));
        if (!matcher.find()) return "Paris";
        String result = matcher.group(1).trim();
        // Fixture-only parser stops at common temporal/prepositional phrases.
        result = result.replaceFirst("(?i)\\s+(next|this|on|for|within|with|from|during)\\b.*$", "").trim();
```

```
        if (result.isBlank()) return "Paris";
        return result.length() > 40 ? result.substring(0, 40) : result;
    }

    private String planFirst(String prompt) {
```

Important boundary
Fixture behavior is deterministic and should not be described as semantic LLM reasoning. It is used so every method call and observation can be reproduced offline.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Procedure-specific prompt and result parsing

The procedure’s code writes an instruction string containing a required operation marker and the allowed action catalog. LlmReasoningAction wraps this instruction with goal, cycle and selected working-memory view. The scripted model returns JSON; ActionOutputParser enforces the expected data structure; the procedure then validates action IDs against the active ActionRegistry.

That layered validation is intentional: prompt wording guides the model, JSON parsing checks the schema, ActionCall checks bounded string values, and ActionRegistry checks whether the specific action exists and accepts those arguments. The action adapter still checks domain preconditions such as calendar.checked before performing simulated inventory search.

Actual source excerpt: dev/coala/core/action/ActionOutputParser.java (source lines 35-70)

```
        requireOnlyFields(root, Set.of("actionId", "arguments"), "action proposal");
        JsonNode actionId = root.get("actionId");
        if (actionId == null || !actionId.isTextual() || actionId.asText().isBlank()) {
            throw new InvalidModelOutputException("Action JSON must contain a non-empty string 'actionId'.");
        }
        return buildActionCall(actionId.asText(), parseArguments(root.get("arguments")));
    }

    /** Parses a set of at least two distinct alternatives. */
    public List<ActionCall> parseCandidates(String raw, int maxCandidates) {
        if (maxCandidates < 2) throw new IllegalArgumentException("maxCandidates must be at least 2");
        JsonNode root = parseJson(raw, "candidate proposal");
        requireObject(root, "Candidate JSON must be an object.");
        requireOnlyFields(root, Set.of("candidates"), "candidate proposal");
        JsonNode candidates = root.get("candidates");
        if (candidates == null || !candidates.isArray() || candidates.size() < 2) {
            throw new InvalidModelOutputException("Candidate JSON must contain at least two alternatives in a
        'candidates' array.");
        }
        if (candidates.size() > maxCandidates) {
            throw new InvalidModelOutputException("Model proposed " + candidates.size()
                    + " candidates; configured limit is " + maxCandidates + ".");
        }
        List<ActionCall> parsed = new ArrayList<>();
        Set<ActionCall> seen = new HashSet<>();
        for (JsonNode candidate : candidates) {
            requireObject(candidate, "Every candidate must be a JSON object.");
            requireOnlyFields(candidate, Set.of("actionId", "arguments"), "candidate action");
            JsonNode id = candidate.get("actionId");
            if (id == null || !id.isTextual() || id.asText().isBlank()) {
                throw new InvalidModelOutputException("Every candidate must contain a non-empty 'actionId'.");
            }
            ActionCall call = buildActionCall(id.asText(), parseArguments(candidate.get("arguments")));
            if (!seen.add(call)) {
                throw new InvalidModelOutputException("Candidate set contains duplicate action calls; propose
        distinct alternatives.");
            }
            parsed.add(call);
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

ActionResult, WorkingMemory and events determine loop progression

An action returns ActionResult with observation, terminal and awaitingInput. AbstractLoopingDecisionProcedure writes latest_action and latest_observation from that result and emits EXECUTION or LEARNING depending on action.kind. If awaitingInput is true, it returns AWAITING_INPUT; if terminal is true, it returns TERMINAL_ACTION; otherwise the next cycle starts until maxCycles is reached.

The public AgentRunResult contains procedure id, goal, final observation, number of cycles, termination reason and events. This result is what DemoUseCaseService wraps with the sessionId and what the controller serializes to JSON.

Actual source excerpt: dev/coala/core/procedure/AbstractLoopingDecisionProcedure.java (source lines 90-121)

```
            context.workingMemory().put("selected_action_id", selected.actionId());
            context.workingMemory().put("selected_action_arguments", selected.arguments());
            try {
                ActionResult result = action.execute(selected, context);
                finalObservation = result.observation();
                context.workingMemory().put("latest_action", selected.actionId());
                context.workingMemory().put("latest_observation", result.observation());
                events.add(new DecisionEvent(cycle, action.kind() == dev.coala.core.ActionKind.LEARNING
                        ? DecisionStage.LEARNING : DecisionStage.EXECUTION,
                        selected.actionId() + " -> " + truncate(result.observation(), 240)));
                if (result.awaitingInput()) {
                    events.add(new DecisionEvent(cycle, DecisionStage.TERMINATION, "Action is awaiting a user or
     external input; resume the interaction in a later request."));
                    return new AgentRunResult(id(), goal, finalObservation, cycle,
                            TerminationReason.AWAITING_INPUT, events);
                }
                if (result.terminal()) {
                    events.add(new DecisionEvent(cycle, DecisionStage.TERMINATION, "Selected action marked the
    run complete."));
                    return new AgentRunResult(id(), goal, finalObservation, cycle,
                            TerminationReason.TERMINAL_ACTION, events);
                }
            } catch (LanguageModelException ex) {
                throw ex;
            } catch (RuntimeException ex) {
                throw new ActionExecutionException("Action '" + selected.actionId() + "' failed at cycle " +
    cycle, ex);
            }
        }

        events.add(new DecisionEvent(maxCycles, DecisionStage.TERMINATION,
                "Safety bound reached: maxCycles=" + maxCycles + "."));
        return new AgentRunResult(id(), goal, finalObservation, maxCycles,
                TerminationReason.MAX_CYCLES_REACHED, events);
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The API return path does not alter the final agent observation

Once CoalaDecisionProcedure.run returns AgentRunResult, CoalaAgent returns that same result after its optional trace/listener hooks. DemoUseCaseService.run creates `new UseCaseRunResponse(session.sessionId(), result)`. CoalaController returns the record and Spring serializes it to JSON.

The finalObservation field is the observation returned by the last selected action. For these examples it is intentionally different by domain: the full evidence brief for research, the itinerary text for travel, and the escalation guidance for support.

State and observable values

Field	Value / meaning
Envelope	UseCaseRunResponse
sessionId	Opaque server-generated value for follow-up calls
run.procedure	Procedure id string
run.cyclesExecuted	2
run.terminationReason	TERMINAL_ACTION
run.finalObservation	Use-case-specific final action observation

Actual source excerpt: dev/coala/demo/UseCaseRunResponse.java (source lines 1-6)

```
package dev.coala.demo;

import dev.coala.core.AgentRunResult;

/** API envelope allowing the caller to resume the same long-term-memory session later. */
public record UseCaseRunResponse(String sessionId, AgentRunResult run) {}
```

Important boundary
Cycle events are explanations of stages, not a complete method profiler. The pages in this document add the finer method-call ledger that events alone do not contain.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 1 / 1 - the loop starts from the latest observation

Cycle 1 starts with User request received: My account is locked and I see an unauthorized payment; help me regain access safely.. The base loop increments the cycle field, reads latest_observation, records an OBSERVATION event, and clears stale plan state. It deliberately does not erase support.triage, because domain results remain available in working memory through the run.

Unlike the deliberative research example, IterativeReasonActDecisionProcedure has no separate retrieval action, proposal set, evaluator, or score policy. It is a repeated one-next-action process: one reasoning call produces one ActionCall, then that registered action executes, and the next cycle sees its observation.

Method-call ledger

01	context.setCycle(1).
02	get latest_observation from WorkingMemory.
03	Append OBSERVATION DecisionEvent.
04	Clear plan.candidates, plan.evaluations, plan.selectedAction, rejection keys, and selected_action fields.
05	Call IterativeReasonActDecisionProcedure.planNextAction(context, actions).

State and observable values

Field	Value / meaning
cycle	1
input observation	User request received: My account is locked and I see an unauthorized payment; help me regain access safely.
decision procedure	iterative-reason-act
stale plan fields	cleared

Actual source excerpt: dev/coala/core/procedure/AbstractLoopingDecisionProcedure.java (source lines 43-61)

```
String observation = context.workingMemory().getString("latest_observation", "");
events.add(new DecisionEvent(cycle, DecisionStage.OBSERVATION,
        truncate(observation, 240)));

// Do not let a previous cycle's plan be mistaken for the current selection.
context.workingMemory().put("plan.candidates", null);
context.workingMemory().put("plan.evaluations", null);
context.workingMemory().put("plan.selectedAction", null);
context.workingMemory().put("plan.selectionRejected", null);
context.workingMemory().put("plan.last_rejection", null);
context.workingMemory().put("plan.previousRejectedCandidates", null);
context.workingMemory().put("selected_action", null);
context.workingMemory().put("selected_action_id", null);
context.workingMemory().put("selected_action_arguments", null);

PlanDecision plan;
try {
    plan = planNextAction(context, actions);
} catch (DecisionProcedureException | LanguageModelException ex) {
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 1 / 2 - build the ITERATIVE_NEXT_ACTION prompt

IterativeReasonActDecisionProcedure.planNextAction constructs an instruction requiring one JSON action call with actionId and arguments. The catalog formatter lists only registered actions and their contracts. The instruction asks the model to use the goal and latest observation and not to claim any tool action has already happened.

The procedure calls reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context). LlmReasoningAction wraps that instruction with operation, cycle, goal, and a bounded working-memory view. For cycle 1, the latest observation is the user-request message; for cycle 2, it is the classification action's observation.

Method-call ledger

01	ActionCatalogFormatter.format(actions.descriptors()).
02	reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context).
03	LlmReasoningAction increments call budget and builds the selected memory view.
04	LanguageModel.generate receives the sanitized prompt.

Actual source excerpt: dev/coala/core/procedure/IterativeReasonActDecisionProcedure.java (source lines 24-45)

```
    @Override
    public String id() { return "iterative-reason-act"; }

    @Override
    protected PlanDecision planNextAction(AgentContext context, ActionRegistry actions) {
        String instruction = "OPERATION ITERATIVE_NEXT_ACTION. Choose exactly one next action as JSON: "
                + "{\"actionId\":\"...\",\"arguments\":{\"key\":\"string value\"}}. "
                + "Available actions and their argument contracts: " +
        actionCatalogFormatter.format(actions.descriptors()) + ". "
                + "Use the goal and latest observation in working memory. Propose only the next action for this
        cycle; "
                + "do not claim that a tool action has already happened. An action can finish the run by
        returning a terminal result. "
                + "Return JSON only.";
        String output = reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context);
        var selected = parser.parseAction(output);
        context.workingMemory().put("reasoning.latest", null);
        return new PlanDecision(selected, List.of(
                new DecisionEvent(context.cycle(), DecisionStage.REASONING, "Reasoned from the current
        observation and working-memory state."),
                new DecisionEvent(context.cycle(), DecisionStage.PROPOSAL, "Proposed one next action for this
        cycle."),
                new DecisionEvent(context.cycle(), DecisionStage.SELECTION, "Selected the single proposed action
         for execution.")));
    }

}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 1 / 3 - the offline model chooses a fixture action

ScriptedLanguageModel.iterativeAction sees both classify_support_case and escalate_case in the prompt catalog and routes to supportAction. In cycle 1, supportAction returns classify_support_case because cycle==1. In cycle 2, it parses the goal and chooses escalate_case because it sees elevated keywords (locked, unauthorized, payment, or security).

The exact scripted output is {"actionId":"classify_support_case","arguments":{}}. This output is a deterministic branch in ScriptedLanguageModel; it is not generated by an LLM, and the branch condition matters for interpreting the demo. The same method has a different branch for travel catalogs and falls back to the coding sequence for the coding catalog.

This method has not executed the chosen action. It has only returned JSON text. The code must still parse it, validate the ActionCall, resolve it through the registered allow-list, and call execute.

Method-call ledger

01	ScriptedLanguageModel.generate(prompt) extracts OPERATION and CYCLE.
02	switch selects ITERATIVE_NEXT_ACTION -> iterativeAction(cycle, prompt).
03	Catalog string match routes to supportAction(cycle, prompt).
04	Cycle 1 returns classify_support_case; cycle 2 checks simple elevated-word list and returns escalate_case.

Actual source excerpt: dev/coala/llm/ScriptedLanguageModel.java (source lines 21-43)

```
        int cycle = extractCycle(prompt);
        return switch (operation) {
            case "DIRECT_ACTION" -> "{\"actionId\":\"respond\",\"arguments\":{\"text\":\"This is the
        direct-action demo response. The request was handled by one selected dialogue grounding action; enable
        the openai profile to use an actual LLM.\"}}";
            case "PLAN_FIRST" -> planFirst(prompt);
            case "ITERATIVE_NEXT_ACTION" -> iterativeAction(cycle, prompt);
            case "PROPOSE_CANDIDATES" -> prompt.contains("format_evidence_brief") ? researchCandidates(cycle,
        prompt) : puzzleCandidates(cycle);
            case "EVALUATE_CANDIDATES" -> prompt.contains("format_evidence_brief") ? researchScores(cycle) :
        puzzleScores(cycle);
            default -> throw new IllegalArgumentException("Scripted adapter has no fixture for operation=" +
        operation);
        };
    }

    private String extractOperation(String prompt) {
        Matcher matcher = Pattern.compile("OPERATION=([A-Z_]+)").matcher(prompt);
        if (!matcher.find()) throw new IllegalArgumentException("Prompt has no OPERATION marker.");
        return matcher.group(1);
    }

    private int extractCycle(String prompt) {
        Matcher matcher = CYCLE.matcher(prompt);
        return matcher.find() ? Integer.parseInt(matcher.group(1)) : 1;
    }

    private String extractGoal(String prompt) {
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 1 / 4 - parse the action JSON and resolve the implementation

The procedure calls ActionOutputParser.parseAction(raw). The parser requires one JSON object with only actionId and arguments, verifies a nonempty string actionId, parses string-valued arguments, and constructs an ActionCall. The procedure then clears reasoning.latest and returns PlanDecision with the single candidate and the REASONING, PROPOSAL and SELECTION events.

After planNextAction returns, AbstractLoopingDecisionProcedure calls actions.validate(selected). This second check resolves the registered object and enforces its contract again. Then it writes plan.selectedAction, selected_action, selected_action_id and selected_action_arguments before action execution.

Method-call ledger

01	ActionOutputParser.parseAction(raw).
02	parseJson -> requireObject -> only fields actionId and arguments.
03	build ActionCall with bounded argument map.
04	WorkingMemory.put("reasoning.latest", null).
05	Construct PlanDecision(selected, one candidate, no scores, stage events).
06	ActionRegistry.validate(selected) -> registered action and argument validation.

Actual source excerpt: dev/coala/core/procedure/IterativeReasonActDecisionProcedure.java (source lines 32-45)

```
                + "Available actions and their argument contracts: " +
        actionCatalogFormatter.format(actions.descriptors()) + ". "
                + "Use the goal and latest observation in working memory. Propose only the next action for this
        cycle; "
                + "do not claim that a tool action has already happened. An action can finish the run by
        returning a terminal result. "
                + "Return JSON only.";
        String output = reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context);
        var selected = parser.parseAction(output);
        context.workingMemory().put("reasoning.latest", null);
        return new PlanDecision(selected, List.of(
                new DecisionEvent(context.cycle(), DecisionStage.REASONING, "Reasoned from the current
        observation and working-memory state."),
                new DecisionEvent(context.cycle(), DecisionStage.PROPOSAL, "Proposed one next action for this
        cycle."),
                new DecisionEvent(context.cycle(), DecisionStage.SELECTION, "Selected the single proposed action
         for execution.")));
    }

}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 1 / 5 - run the selected support action

The selected action is classify_support_case. It has no required arguments, so the empty arguments map satisfies the action contract. The action implementation below reads only the goal/working memory it needs, returns a safe observation and does not call a ticketing or identity system.

The action result is: Mock triage recorded category=account-access, severity=elevated. No user account was accessed.

The classifier lowercases the goal, checks whether it contains any of locked/compromised/fraud/unauthorized/payment/security, and separately checks account-access terms. The exact goal hits both categories, so it stores account-access/elevated.

State and observable values

Field	Value / meaning
support.triage	{category: account-access, severity: elevated, source: deterministic-demo-rule}
latest_action	classify_support_case
latest_observation	Mock triage recorded category=account-access, severity=elevated. No user account was accessed.

Actual source excerpt: dev/coala/demo/action/SupportTriageActions.java (source lines 15-30)

```
    public static GroundingAction classifyCase() {
        return new GroundingAction() {
            @Override public String id() { return "classify_support_case"; }
            @Override public String description() { return "Classify the user's support request using a
        deterministic teaching fixture."; }
            @Override public ActionRisk risk() { return ActionRisk.LOW; }
            @Override public boolean idempotent() { return true; }
            @Override public ActionResult execute(ActionCall call, AgentContext context) {
                String goal = context.workingMemory().getString("goal", "").toLowerCase(java.util.Locale.ROOT);
                boolean elevated = containsAny(goal, "locked", "compromised", "fraud", "unauthorized",
        "payment", "security");
                String category = containsAny(goal, "sign in", "login", "password", "locked") ? "account-access"
         : "general-support";
                context.workingMemory().put("support.triage", java.util.Map.of("category", category,
                        "severity", elevated ? "elevated" : "normal", "source", "deterministic-demo-rule"));
                return ActionResult.continueWith("Mock triage recorded category=" + category + ", severity=" +
        (elevated ? "elevated" : "normal") + ". No user account was accessed.");
            }
        };
    }
```

Important boundary
The action is a deterministic mock. Treat its output as an illustrative workflow response, not a real account assessment or real escalation.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 1 / 6 - store observation and decide whether to continue

The base loop receives ActionResult, writes latest_action and latest_observation, and appends an EXECUTION DecisionEvent because both selected actions are GroundingAction. It checks awaitingInput first; both results have awaitingInput=false. It then inspects terminal.

Cycle 1 result: nonterminal. The loop begins cycle 2 with the classification observation. That new cycle constructs a new ITERATIVE_NEXT_ACTION prompt and invokes the scripted adapter again.

Note that DecisionEvent records stages and short summaries, not a complete call-profiler. The detailed method ledger in this PDF shows the nested calls that the event list intentionally compresses.

Method-call ledger

01	ActionResult returned.
02	WorkingMemory.put("latest_action", selected.actionId()).
03	WorkingMemory.put("latest_observation", result.observation()).
04	Append EXECUTION event.
05	result.awaitingInput? false.
06	result.terminal? false at cycle 1, true at cycle 2.
07	Return result on cycle 2; otherwise loop.

State and observable values

Field	Value / meaning
cycles executed	1
selected	classify_support_case
termination	not yet terminal
next state	enter cycle 2

Actual source excerpt: dev/coala/core/procedure/AbstractLoopingDecisionProcedure.java (source lines 92-121)

```
            try {
                ActionResult result = action.execute(selected, context);
                finalObservation = result.observation();
                context.workingMemory().put("latest_action", selected.actionId());
                context.workingMemory().put("latest_observation", result.observation());
                events.add(new DecisionEvent(cycle, action.kind() == dev.coala.core.ActionKind.LEARNING
                        ? DecisionStage.LEARNING : DecisionStage.EXECUTION,
                        selected.actionId() + " -> " + truncate(result.observation(), 240)));
                if (result.awaitingInput()) {
                    events.add(new DecisionEvent(cycle, DecisionStage.TERMINATION, "Action is awaiting a user or
     external input; resume the interaction in a later request."));
                    return new AgentRunResult(id(), goal, finalObservation, cycle,
                            TerminationReason.AWAITING_INPUT, events);
                }
                if (result.terminal()) {
                    events.add(new DecisionEvent(cycle, DecisionStage.TERMINATION, "Selected action marked the
        run complete."));
                    return new AgentRunResult(id(), goal, finalObservation, cycle,
                            TerminationReason.TERMINAL_ACTION, events);
                }
            } catch (LanguageModelException ex) {
                throw ex;
            } catch (RuntimeException ex) {
                throw new ActionExecutionException("Action '" + selected.actionId() + "' failed at cycle " +
        cycle, ex);
            }
        }

        events.add(new DecisionEvent(maxCycles, DecisionStage.TERMINATION,
                "Safety bound reached: maxCycles=" + maxCycles + "."));
        return new AgentRunResult(id(), goal, finalObservation, maxCycles,
                TerminationReason.MAX_CYCLES_REACHED, events);
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 2 / 1 - the loop starts from the latest observation

Cycle 2 starts with Mock triage recorded category=account-access, severity=elevated. No user account was accessed.. The base loop increments the cycle field, reads latest_observation, records an OBSERVATION event, and clears stale plan state. It deliberately does not erase support.triage, because domain results remain available in working memory through the run.

Unlike the deliberative research example, IterativeReasonActDecisionProcedure has no separate retrieval action, proposal set, evaluator, or score policy. It is a repeated one-next-action process: one reasoning call produces one ActionCall, then that registered action executes, and the next cycle sees its observation.

Method-call ledger

01	context.setCycle(2).
02	get latest_observation from WorkingMemory.
03	Append OBSERVATION DecisionEvent.
04	Clear plan.candidates, plan.evaluations, plan.selectedAction, rejection keys, and selected_action fields.
05	Call IterativeReasonActDecisionProcedure.planNextAction(context, actions).

State and observable values

Field	Value / meaning
cycle	2
input observation	Mock triage recorded category=account-access, severity=elevated. No user account was accessed.
decision procedure	iterative-reason-act
stale plan fields	cleared

Actual source excerpt: dev/coala/core/procedure/AbstractLoopingDecisionProcedure.java (source lines 43-61)

```
String observation = context.workingMemory().getString("latest_observation", "");
events.add(new DecisionEvent(cycle, DecisionStage.OBSERVATION,
        truncate(observation, 240)));

// Do not let a previous cycle's plan be mistaken for the current selection.
context.workingMemory().put("plan.candidates", null);
context.workingMemory().put("plan.evaluations", null);
context.workingMemory().put("plan.selectedAction", null);
context.workingMemory().put("plan.selectionRejected", null);
context.workingMemory().put("plan.last_rejection", null);
context.workingMemory().put("plan.previousRejectedCandidates", null);
context.workingMemory().put("selected_action", null);
context.workingMemory().put("selected_action_id", null);
context.workingMemory().put("selected_action_arguments", null);

PlanDecision plan;
try {
    plan = planNextAction(context, actions);
} catch (DecisionProcedureException | LanguageModelException ex) {
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 2 / 2 - build the ITERATIVE_NEXT_ACTION prompt

IterativeReasonActDecisionProcedure.planNextAction constructs an instruction requiring one JSON action call with actionId and arguments. The catalog formatter lists only registered actions and their contracts. The instruction asks the model to use the goal and latest observation and not to claim any tool action has already happened.

The procedure calls reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context). LlmReasoningAction wraps that instruction with operation, cycle, goal, and a bounded working-memory view. For cycle 1, the latest observation is the user-request message; for cycle 2, it is the classification action's observation.

Method-call ledger

01	ActionCatalogFormatter.format(actions.descriptors()).
02	reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context).
03	LlmReasoningAction increments call budget and builds the selected memory view.
04	LanguageModel.generate receives the sanitized prompt.

Actual source excerpt: dev/coala/core/procedure/IterativeReasonActDecisionProcedure.java (source lines 24-45)

```
    @Override
    public String id() { return "iterative-reason-act"; }

    @Override
    protected PlanDecision planNextAction(AgentContext context, ActionRegistry actions) {
        String instruction = "OPERATION ITERATIVE_NEXT_ACTION. Choose exactly one next action as JSON: "
                + "{\"actionId\":\"...\",\"arguments\":{\"key\":\"string value\"}}. "
                + "Available actions and their argument contracts: " +
        actionCatalogFormatter.format(actions.descriptors()) + ". "
                + "Use the goal and latest observation in working memory. Propose only the next action for this
        cycle; "
                + "do not claim that a tool action has already happened. An action can finish the run by
        returning a terminal result. "
                + "Return JSON only.";
        String output = reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context);
        var selected = parser.parseAction(output);
        context.workingMemory().put("reasoning.latest", null);
        return new PlanDecision(selected, List.of(
                new DecisionEvent(context.cycle(), DecisionStage.REASONING, "Reasoned from the current
        observation and working-memory state."),
                new DecisionEvent(context.cycle(), DecisionStage.PROPOSAL, "Proposed one next action for this
        cycle."),
                new DecisionEvent(context.cycle(), DecisionStage.SELECTION, "Selected the single proposed action
         for execution.")));
    }

}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 2 / 3 - the offline model chooses a fixture action

ScriptedLanguageModel.iterativeAction sees both classify_support_case and escalate_case in the prompt catalog and routes to supportAction. In cycle 1, supportAction returns classify_support_case because cycle==1. In cycle 2, it parses the goal and chooses escalate_case because it sees elevated keywords (locked, unauthorized, payment, or security).

The exact scripted output is {"actionId":"escalate_case","arguments":{}}. This output is a deterministic branch in ScriptedLanguageModel; it is not generated by an LLM, and the branch condition matters for interpreting the demo. The same method has a different branch for travel catalogs and falls back to the coding sequence for the coding catalog.

This method has not executed the chosen action. It has only returned JSON text. The code must still parse it, validate the ActionCall, resolve it through the registered allow-list, and call execute.

Method-call ledger

01	ScriptedLanguageModel.generate(prompt) extracts OPERATION and CYCLE.
02	switch selects ITERATIVE_NEXT_ACTION -> iterativeAction(cycle, prompt).
03	Catalog string match routes to supportAction(cycle, prompt).
04	Cycle 1 returns classify_support_case; cycle 2 checks simple elevated-word list and returns escalate_case.

Actual source excerpt: dev/coala/llm/ScriptedLanguageModel.java (source lines 21-43)

```
        int cycle = extractCycle(prompt);
        return switch (operation) {
            case "DIRECT_ACTION" -> "{\"actionId\":\"respond\",\"arguments\":{\"text\":\"This is the
        direct-action demo response. The request was handled by one selected dialogue grounding action; enable
        the openai profile to use an actual LLM.\"}}";
            case "PLAN_FIRST" -> planFirst(prompt);
            case "ITERATIVE_NEXT_ACTION" -> iterativeAction(cycle, prompt);
            case "PROPOSE_CANDIDATES" -> prompt.contains("format_evidence_brief") ? researchCandidates(cycle,
        prompt) : puzzleCandidates(cycle);
            case "EVALUATE_CANDIDATES" -> prompt.contains("format_evidence_brief") ? researchScores(cycle) :
        puzzleScores(cycle);
            default -> throw new IllegalArgumentException("Scripted adapter has no fixture for operation=" +
        operation);
        };
    }

    private String extractOperation(String prompt) {
        Matcher matcher = Pattern.compile("OPERATION=([A-Z_]+)").matcher(prompt);
        if (!matcher.find()) throw new IllegalArgumentException("Prompt has no OPERATION marker.");
        return matcher.group(1);
    }

    private int extractCycle(String prompt) {
        Matcher matcher = CYCLE.matcher(prompt);
        return matcher.find() ? Integer.parseInt(matcher.group(1)) : 1;
    }

    private String extractGoal(String prompt) {
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 2 / 4 - parse the action JSON and resolve the implementation

The procedure calls ActionOutputParser.parseAction(raw). The parser requires one JSON object with only actionId and arguments, verifies a nonempty string actionId, parses string-valued arguments, and constructs an ActionCall. The procedure then clears reasoning.latest and returns PlanDecision with the single candidate and the REASONING, PROPOSAL and SELECTION events.

After planNextAction returns, AbstractLoopingDecisionProcedure calls actions.validate(selected). This second check resolves the registered object and enforces its contract again. Then it writes plan.selectedAction, selected_action, selected_action_id and selected_action_arguments before action execution.

Method-call ledger

01	ActionOutputParser.parseAction(raw).
02	parseJson -> requireObject -> only fields actionId and arguments.
03	build ActionCall with bounded argument map.
04	WorkingMemory.put("reasoning.latest", null).
05	Construct PlanDecision(selected, one candidate, no scores, stage events).
06	ActionRegistry.validate(selected) -> registered action and argument validation.

Actual source excerpt: dev/coala/core/procedure/IterativeReasonActDecisionProcedure.java (source lines 32-45)

```
                + "Available actions and their argument contracts: " +
        actionCatalogFormatter.format(actions.descriptors()) + ". "
                + "Use the goal and latest observation in working memory. Propose only the next action for this
        cycle; "
                + "do not claim that a tool action has already happened. An action can finish the run by
        returning a terminal result. "
                + "Return JSON only.";
        String output = reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context);
        var selected = parser.parseAction(output);
        context.workingMemory().put("reasoning.latest", null);
        return new PlanDecision(selected, List.of(
                new DecisionEvent(context.cycle(), DecisionStage.REASONING, "Reasoned from the current
        observation and working-memory state."),
                new DecisionEvent(context.cycle(), DecisionStage.PROPOSAL, "Proposed one next action for this
        cycle."),
                new DecisionEvent(context.cycle(), DecisionStage.SELECTION, "Selected the single proposed action
         for execution.")));
    }

}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 2 / 5 - run the selected support action

The selected action is escalate_case. It has no required arguments, so the empty arguments map satisfies the action contract. The action implementation below reads only the goal/working memory it needs, returns a safe observation and does not call a ticketing or identity system.

The action result is: Escalation recommendation: use the provider's verified support channel for account-access review. Demo reference SUP-DEMO-1042 is fictional; no real support ticket, identity check or account action occurred.

The escalation action returns a terminal recommendation with a fictional demo reference. It does not read support.triage, make a network call, or create a real case.

State and observable values

Field	Value / meaning
support.triage	Still present in per-run WorkingMemory; not the fixture’s selection input
latest_action	escalate_case
latest_observation	Final escalation recommendation
terminalReason	TERMINAL_ACTION

Actual source excerpt: dev/coala/demo/action/SupportTriageActions.java (source lines 44-54)

```
    public static GroundingAction escalateCase() {
        return new GroundingAction() {
            @Override public String id() { return "escalate_case"; }
            @Override public String description() { return "Return a simulated recommendation to escalate an
        elevated-risk support case; no real ticket is created."; }
            @Override public ActionRisk risk() { return ActionRisk.HIGH; }
            @Override public boolean idempotent() { return true; }
            @Override public ActionResult execute(ActionCall call, AgentContext context) {
                return ActionResult.finish("Escalation recommendation: use the provider's verified support
        channel for account-access review. Demo reference SUP-DEMO-1042 is fictional; no real support ticket,
        identity check or account action occurred.");
            }
        };
    }
```

Important boundary
The action is a deterministic mock. Treat its output as an illustrative workflow response, not a real account assessment or real escalation.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Cycle 2 / 6 - store observation and decide whether to continue

The base loop receives ActionResult, writes latest_action and latest_observation, and appends an EXECUTION DecisionEvent because both selected actions are GroundingAction. It checks awaitingInput first; both results have awaitingInput=false. It then inspects terminal.

Cycle 2 result: terminal. The loop appends a TERMINATION event and returns AgentRunResult with cyclesExecuted=2 and TERMINAL_ACTION.

Note that DecisionEvent records stages and short summaries, not a complete call-profiler. The detailed method ledger in this PDF shows the nested calls that the event list intentionally compresses.

Method-call ledger

01	ActionResult returned.
02	WorkingMemory.put("latest_action", selected.actionId()).
03	WorkingMemory.put("latest_observation", result.observation()).
04	Append EXECUTION event.
05	result.awaitingInput? false.
06	result.terminal? false at cycle 1, true at cycle 2.
07	Return result on cycle 2; otherwise loop.

State and observable values

Field	Value / meaning
cycles executed	2
selected	escalate_case
termination	TERMINAL_ACTION
next state	return to API

Actual source excerpt: dev/coala/core/procedure/AbstractLoopingDecisionProcedure.java (source lines 92-121)

```
            try {
                ActionResult result = action.execute(selected, context);
                finalObservation = result.observation();
                context.workingMemory().put("latest_action", selected.actionId());
                context.workingMemory().put("latest_observation", result.observation());
                events.add(new DecisionEvent(cycle, action.kind() == dev.coala.core.ActionKind.LEARNING
                        ? DecisionStage.LEARNING : DecisionStage.EXECUTION,
                        selected.actionId() + " -> " + truncate(result.observation(), 240)));
                if (result.awaitingInput()) {
                    events.add(new DecisionEvent(cycle, DecisionStage.TERMINATION, "Action is awaiting a user or
     external input; resume the interaction in a later request."));
                    return new AgentRunResult(id(), goal, finalObservation, cycle,
                            TerminationReason.AWAITING_INPUT, events);
                }
                if (result.terminal()) {
                    events.add(new DecisionEvent(cycle, DecisionStage.TERMINATION, "Selected action marked the
        run complete."));
                    return new AgentRunResult(id(), goal, finalObservation, cycle,
                            TerminationReason.TERMINAL_ACTION, events);
                }
            } catch (LanguageModelException ex) {
                throw ex;
            } catch (RuntimeException ex) {
                throw new ActionExecutionException("Action '" + selected.actionId() + "' failed at cycle " +
        cycle, ex);
            }
        }

        events.add(new DecisionEvent(maxCycles, DecisionStage.TERMINATION,
                "Safety bound reached: maxCycles=" + maxCycles + "."));
        return new AgentRunResult(id(), goal, finalObservation, maxCycles,
                TerminationReason.MAX_CYCLES_REACHED, events);
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Why the classification action produces account-access / elevated

SupportTriageActions.classifyCase reads `goal` from working memory and calls toLowerCase(Locale.ROOT). containsAny compares the entire normalized string using String.contains against a finite list. The exact sample includes “locked” and “unauthorized payment”, so elevated=true. It also includes “locked”, so the account-access category wins over general-support.

The action stores a small declarative map under support.triage. Because this is a plain in-memory value, later actions in the same run can read it. The action returns a summary observation rather than dumping the whole state object; this keeps the observation concise and clarifies that no account was accessed.

State and observable values

Field	Value / meaning
goal contains locked	yes
goal contains unauthorized	yes
elevated	true
category	account-access
source	deterministic-demo-rule

Actual source excerpt: dev/coala/demo/action/SupportTriageActions.java (source lines 21-30)

```
            @Override public ActionResult execute(ActionCall call, AgentContext context) {
                String goal = context.workingMemory().getString("goal", "").toLowerCase(java.util.Locale.ROOT);
                boolean elevated = containsAny(goal, "locked", "compromised", "fraud", "unauthorized",
        "payment", "security");
                String category = containsAny(goal, "sign in", "login", "password", "locked") ? "account-access"
         : "general-support";
                context.workingMemory().put("support.triage", java.util.Map.of("category", category,
                        "severity", elevated ? "elevated" : "normal", "source", "deterministic-demo-rule"));
                return ActionResult.continueWith("Mock triage recorded category=" + category + ", severity=" +
        (elevated ? "elevated" : "normal") + ". No user account was accessed.");
            }
        };
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The helper containsAny is simple string matching, not a trained classifier

The helper iterates each provided needle and returns true on the first `text.contains(needle)` match. It does not understand synonyms, context, negation, entities, false positives, or the severity of a payment event. It will flag a phrase like “I am worried about fraud” just as it flags a confirmed fraudulent transaction.

This is intentionally inspectable code for the simulation. If production triage uses a model, define a typed classification result, validate allowed category/severity values, preserve evidence, and keep actual escalation behind an authorized, auditable action. Do not silently promote this demo’s keyword list into a general support classifier.

Actual source excerpt: dev/coala/demo/action/SupportTriageActions.java (source lines 55-60)

```
    private static boolean containsAny(String text, String... needles) {
        for (String needle : needles) if (text.contains(needle)) return true;
        return false;
    }
}
```

Important boundary
The goal is user text; containsAny is a brittle demonstration rule. The guide calls out this limitation instead of describing it as general intelligence.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Important: the scripted next-action choice does not read support.triage

The source of the cycle-2 choice is ScriptedLanguageModel.supportAction. It extracts the goal from the model prompt, lowercases it, recomputes the same high-level elevated-keyword condition, and returns escalate_case or suggest_safe_steps. It does not inspect the serialized support.triage working-memory object before it decides.

Therefore the simulation demonstrates two cycles and shows the classification observation becoming the next observation, but it does not demonstrate a causal dependency where the second choice consumes the structured triage output. A real provider could reason over that state when included in the prompt, but this fixture is deterministic and goal-string driven.

Actual source excerpt: dev/coala/llm/ScriptedLanguageModel.java (source lines 80-103)

```
        String destination = jsonEscape(extractDestination(prompt));
        return switch (cycle) {
            case 1 -> "{\"actionId\":\"check_calendar\",\"arguments\":{}}";
            case 2 -> "{\"actionId\":\"search_flights\",\"arguments\":{\"destination\":\"" + destination +
        "\",\"origin\":\"YUL\"}}";
            case 3 -> "{\"actionId\":\"search_hotels\",\"arguments\":{\"destination\":\"" + destination +
        "\",\"checkIn\":\"2026-11-05\",\"nights\":\"3\"}}";
            case 4 -> "{\"actionId\":\"draft_itinerary\",\"arguments\":{}}";
            case 5 -> "{\"actionId\":\"learn_episode\",\"arguments\":{\"content\":\"Travel workflow lesson:
        check the calendar before inventory searches and never book from mock
        results.\",\"success\":\"true\",\"importance\":\"0.6\",\"source\":\"travel-demo\"}}";
            default -> "{\"actionId\":\"present_itinerary\",\"arguments\":{}}";
        };
    }

    private String extractBudget(String prompt) {
        Matcher matcher = BUDGET.matcher(extractGoal(prompt));
        return matcher.find() ? matcher.group(1) : "1400";
    }

    private String supportAction(int cycle, String prompt) {
        if (cycle == 1) return "{\"actionId\":\"classify_support_case\",\"arguments\":{}}";
        String goal = extractGoal(prompt).toLowerCase(Locale.ROOT);
        boolean elevated = containsAny(goal, "locked", "compromised", "fraud", "unauthorized", "payment",
        "security");
        return elevated
                ? "{\"actionId\":\"escalate_case\",\"arguments\":{}}"
                : "{\"actionId\":\"suggest_safe_steps\",\"arguments\":{}}";
    }
```

Important boundary
Do not over-read a demo trace. In a future production implementation, add a test that changes the stored classification while holding the goal constant and verify the planning behavior changes as designed.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The alternative branch: safe guidance when the elevated rule is false

If the goal were “I forgot my password and cannot sign in”, cycle 1 still selects classify_support_case, because the fixture always classifies first. The classifier would set category=account-access but elevated=false, because its elevated keyword list does not include password or sign in. The second scripted call would select suggest_safe_steps.

The safe-steps action returns a terminal message advising official recovery, no password/one-time-code sharing, and verified support contact if access remains blocked. The branch demonstrates how one iterative procedure can choose different terminal actions. It remains a fixture: the action text is fixed and not tailored using the structured category.

State and observable values

Field	Value / meaning
sample alternate goal	I forgot my password and cannot sign in.
category	account-access
elevated	false
cycle 2 selected action	suggest_safe_steps
cycles to finish	2

Actual source excerpt: dev/coala/demo/action/SupportTriageActions.java (source lines 32-42)

```
    public static GroundingAction suggestSafeSteps() {
        return new GroundingAction() {
            @Override public String id() { return "suggest_safe_steps"; }
            @Override public String description() { return "Give safe, non-invasive support guidance without
        changing an account."; }
            @Override public ActionRisk risk() { return ActionRisk.LOW; }
            @Override public boolean idempotent() { return true; }
            @Override public ActionResult execute(ActionCall call, AgentContext context) {
                return ActionResult.finish("Suggested next steps (demo): use the service's official recovery
        path, do not share passwords or one-time codes, and avoid repeatedly retrying if the service says the
        account is locked. If access remains blocked, contact the provider's verified support channel. No
        account was inspected or changed.");
            }
        };
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Why risk metadata alone does not enforce authorization

The action descriptors label classify_support_case and suggest_safe_steps as LOW risk and escalate_case as HIGH risk. Risk is metadata for planning and display; the action implementation shown does not call an external ticket system and does not itself consult a separate human-approval service.

If this same action ID later points to a real case-management integration, the implementation must preserve an explicit authorization boundary: validate current principal and tenant, enforce policy at the server-side action boundary, capture approval if required, make retries idempotent where possible, and emit an auditable event. A model's choice of a high-risk action must never be treated as authorization.

Actual source excerpt: dev/coala/demo/action/SupportTriageActions.java (source lines 44-54)

```
    public static GroundingAction escalateCase() {
        return new GroundingAction() {
            @Override public String id() { return "escalate_case"; }
            @Override public String description() { return "Return a simulated recommendation to escalate an
        elevated-risk support case; no real ticket is created."; }
            @Override public ActionRisk risk() { return ActionRisk.HIGH; }
            @Override public boolean idempotent() { return true; }
            @Override public ActionResult execute(ActionCall call, AgentContext context) {
                return ActionResult.finish("Escalation recommendation: use the provider's verified support
        channel for account-access review. Demo reference SUP-DEMO-1042 is fictional; no real support ticket,
        identity check or account action occurred.");
            }
        };
    }
```

Important boundary
No real escalation is created in the current demo. The high-risk label is descriptive metadata, not proof that authorization enforcement is active in this path.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Working-memory timeline for support, field by field

WorkingMemory begins with goal and latest_observation. The initial cycle has no support.triage key. During cycle 1, the selected_action* fields are written before execution; classifyCase adds support.triage; then the base loop sets latest_action and latest_observation. Cycle 2 clears plan and selected-action fields but retains goal and support.triage, then writes a new selected_action for escalation.

After cycle 2, the final observation is the action’s escalation text. The current `UseCaseRunResponse` includes AgentRunResult.events, but it does not serialize the entire WorkingMemory map, so the triage object is an internal state item rather than an exposed field in the response envelope.

State and observable values

Field	Value / meaning
Before cycle 1	goal; initial latest_observation
After classify	support.triage; latest_action; classification observation
Cycle 2 pre-plan	goal + classification observation + retained support.triage
After escalation	latest_action=escalate_case; latest_observation=final recommendation
Response contains	finalObservation, cycles, termination reason, events, sessionId

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The method-call trace for a successful support request

This is the condensed call sequence for the whole request. The earlier pages explain every step and the selected action bodies. Pay attention to the boundary between model text and executable code: the JSON string is parsed first, resolved against the allow-list second, and only then passed to execute.

The key cross-cycle transition is `latest_observation`: cycle 1's ActionResult becomes the context presented to the procedure in cycle 2. In the scripted implementation, the next action is nevertheless chosen by a fixture branch based on the goal. This difference between the conceptual loop and the fixture's internals is deliberately visible.

Method-call ledger

01	ApiGuardFilter.doFilterInternal -> filterChain.doFilter.
02	CoalaController.run -> DemoUseCaseService.run -> SessionMemoryStore.acquire.
03	runSupport -> runAgent -> AgentContextFactory.scoped -> CoalaAgent.run.
04	AbstractLoopingDecisionProcedure.run(cycle=1) -> IterativeReasonActDecisionProcedure.planNextAction.
05	LlmReasoningAction.reason -> ScriptedLanguageModel.generate -> supportAction -> parseAction.
06	ActionRegistry.validate -> classifyCase.execute -> ActionResult.continueWith.
07	Loop writes observation; cycle 2 reasoning/model/parser/validation repeats for escalate_case.
08	escalateCase.execute -> ActionResult.finish -> AgentRunResult -> UseCaseRunResponse -> JSON response.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Why this procedure makes one reasoning call per cycle

IterativeReasonActDecisionProcedure's `planNextAction` formats the catalog, creates one-next-action instruction, invokes reasoning.reason exactly once, calls parser.parseAction once, clears reasoning.latest, and returns PlanDecision with one candidate and no evaluation score. It does not create a candidate array, call CandidateEvaluator, or call a search algorithm.

The public cycle count counts action executions, not internal LLM operations. Here cycle 1 includes one LLM action proposal then one action execution; cycle 2 includes one proposal then one terminal action execution. If an OpenAI model were configured, each cycle would normally contain one provider call for next-action reasoning.

Method-call ledger

01	format action catalog.
02	ReasoningAction.reason("ITERATIVE_NEXT_ACTION", ...).
03	LLM adapter creates prompt and calls LanguageModel.generate.
04	ActionOutputParser.parseAction(raw).
05	return PlanDecision(single selected call, single candidate, no scores).

Actual source excerpt: dev/coala/core/procedure/IterativeReasonActDecisionProcedure.java (source lines 24-45)

```
    @Override
    public String id() { return "iterative-reason-act"; }

    @Override
    protected PlanDecision planNextAction(AgentContext context, ActionRegistry actions) {
        String instruction = "OPERATION ITERATIVE_NEXT_ACTION. Choose exactly one next action as JSON: "
                + "{\"actionId\":\"...\",\"arguments\":{\"key\":\"string value\"}}. "
                + "Available actions and their argument contracts: " +
        actionCatalogFormatter.format(actions.descriptors()) + ". "
                + "Use the goal and latest observation in working memory. Propose only the next action for this
        cycle; "
                + "do not claim that a tool action has already happened. An action can finish the run by
        returning a terminal result. "
                + "Return JSON only.";
        String output = reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context);
        var selected = parser.parseAction(output);
        context.workingMemory().put("reasoning.latest", null);
        return new PlanDecision(selected, List.of(
                new DecisionEvent(context.cycle(), DecisionStage.REASONING, "Reasoned from the current
        observation and working-memory state."),
                new DecisionEvent(context.cycle(), DecisionStage.PROPOSAL, "Proposed one next action for this
        cycle."),
                new DecisionEvent(context.cycle(), DecisionStage.SELECTION, "Selected the single proposed action
         for execution.")));
    }

}
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

No retrieval component is called in the support path

Although the broader framework contains MemoryRetrievalAction, runSupport does not inject it into this decision procedure. The support procedure's constructor only requires ReasoningAction and ActionOutputParser. It uses the selected prompt-context policy to include approved working-memory keys; it does not query long-term memory as a distinct action in planning.

This is intentional for the narrow teaching scenario. CoALA permits different procedures to use internal actions differently; it would be misleading to show retrieval in the method trace when no call exists. If support needed previous incidents, product policy or account history, add a retrieval operation to the procedure and tests proving the right namespace is used.

Actual source excerpt: dev/coala/core/procedure/IterativeReasonActDecisionProcedure.java (source lines 15-45)

```
public final class IterativeReasonActDecisionProcedure extends AbstractLoopingDecisionProcedure {
    private final ReasoningAction reasoning;
    private final ActionOutputParser parser;
    private final ActionCatalogFormatter actionCatalogFormatter = new ActionCatalogFormatter();

    public IterativeReasonActDecisionProcedure(ReasoningAction reasoning, ActionOutputParser parser) {
        this.reasoning = reasoning;
        this.parser = parser;
    }

    @Override
    public String id() { return "iterative-reason-act"; }

    @Override
    protected PlanDecision planNextAction(AgentContext context, ActionRegistry actions) {
        String instruction = "OPERATION ITERATIVE_NEXT_ACTION. Choose exactly one next action as JSON: "
                + "{\"actionId\":\"...\",\"arguments\":{\"key\":\"string value\"}}. "
                + "Available actions and their argument contracts: " +
        actionCatalogFormatter.format(actions.descriptors()) + ". "
                + "Use the goal and latest observation in working memory. Propose only the next action for this
        cycle; "
                + "do not claim that a tool action has already happened. An action can finish the run by
        returning a terminal result. "
                + "Return JSON only.";
        String output = reasoning.reason("ITERATIVE_NEXT_ACTION", instruction, context);
        var selected = parser.parseAction(output);
        context.workingMemory().put("reasoning.latest", null);
        return new PlanDecision(selected, List.of(
                new DecisionEvent(context.cycle(), DecisionStage.REASONING, "Reasoned from the current
        observation and working-memory state."),
                new DecisionEvent(context.cycle(), DecisionStage.PROPOSAL, "Proposed one next action for this
        cycle."),
                new DecisionEvent(context.cycle(), DecisionStage.SELECTION, "Selected the single proposed action
         for execution.")));
    }

}
```

Important boundary
The current default prompt context allow-list includes support.triage, but the scripted support decision logic does not consume that field when selecting cycle 2. Prompt availability and actual use are different claims.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

The terminal action has a different action implementation, not a different loop

The loop does not special-case support or the word “escalate”. Both selected actions implement GroundingAction. `classify_support_case` returns continueWith, so the base procedure writes its observation and continues. `escalate_case` returns finish, so the same base procedure writes the final observation and exits with TERMINAL_ACTION.

That illustrates how use cases plug different meanings into the same execution shell: the core only inspects `ActionResult.awaitingInput()` and `ActionResult.terminal()`, not the action’s domain-specific string.

Actual source excerpt: dev/coala/core/procedure/AbstractLoopingDecisionProcedure.java (source lines 92-121)

```
            try {
                ActionResult result = action.execute(selected, context);
                finalObservation = result.observation();
                context.workingMemory().put("latest_action", selected.actionId());
                context.workingMemory().put("latest_observation", result.observation());
                events.add(new DecisionEvent(cycle, action.kind() == dev.coala.core.ActionKind.LEARNING
                        ? DecisionStage.LEARNING : DecisionStage.EXECUTION,
                        selected.actionId() + " -> " + truncate(result.observation(), 240)));
                if (result.awaitingInput()) {
                    events.add(new DecisionEvent(cycle, DecisionStage.TERMINATION, "Action is awaiting a user or
         external input; resume the interaction in a later request."));
                    return new AgentRunResult(id(), goal, finalObservation, cycle,
                            TerminationReason.AWAITING_INPUT, events);
                }
                if (result.terminal()) {
                    events.add(new DecisionEvent(cycle, DecisionStage.TERMINATION, "Selected action marked the
        run complete."));
                    return new AgentRunResult(id(), goal, finalObservation, cycle,
                            TerminationReason.TERMINAL_ACTION, events);
                }
            } catch (LanguageModelException ex) {
                throw ex;
            } catch (RuntimeException ex) {
                throw new ActionExecutionException("Action '" + selected.actionId() + "' failed at cycle " +
        cycle, ex);
            }
        }

        events.add(new DecisionEvent(maxCycles, DecisionStage.TERMINATION,
                "Safety bound reached: maxCycles=" + maxCycles + "."));
        return new AgentRunResult(id(), goal, finalObservation, maxCycles,
                TerminationReason.MAX_CYCLES_REACHED, events);
    }
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

What to test when replacing the scripted adapter

A replacement provider must be tested against action-contract failures, invalid JSON, empty responses, and a model choosing a high-risk action. For support, tests should check classification categories and severity against an explicit decision table, not only one sample phrase. Add a causal test: hold the goal constant, change support.triage, and verify the planner behaves as intended if structured triage is supposed to influence the next action.

The current API request does not include a principal with authorization claims. Therefore production escalation should not directly create a ticket from model output unless the action performs its own authorization, idempotency, approval and audit checks.

Important boundary
These are production integration requirements, not claims about current mock action behavior.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Final outcome: what the caller actually receives

A safe escalation recommendation that tells the user to use the provider verified support channel. SUP-DEMO-1042 is fictional. There is no ticket creation, identity verification, account lookup, or payment reversal.

The response envelope is UseCaseRunResponse(sessionId, AgentRunResult). Spring/Jackson serializes the record fields; the controller does not rewrite or summarize the agent’s final observation. The public event stream shows high-level stages and short observation summaries, while the full finalObservation preserves the final action output.

State and observable values

Field	Value / meaning
procedure	iterative-reason-act
cyclesExecuted	2
terminationReason	TERMINAL_ACTION
final action	escalate_case
session id	UUID returned by the service
event list	observation and planning stages plus execution/learning and termination

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Read the event list as a stage trace, not a method profiler

DecisionEvent has cycle, stage and summary. The base loop records OBSERVATION and EXECUTION/LEARNING, and each procedure contributes its own planning events. Research explicitly contributes RETRIEVAL, REASONING, PROPOSAL, EVALUATION and SELECTION stages. Travel records proposal/selection for each plan step; the JSON plan-generation internal model call is not a separate public event object. Support records REASONING, PROPOSAL and SELECTION for each next-action selection.

The event summary is intentionally truncated before inclusion, and the CoalaAgent trace-listener interface is unused by this demo setup. That is why this guide expands the internal method calls beyond the API event list.

Actual source excerpt: dev/coala/core/DecisionEvent.java (source lines 1-7)

```
package dev.coala.core;

public record DecisionEvent(int cycle, DecisionStage stage, String summary) {
    public DecisionEvent {
        summary = summary == null ? "" : summary;
    }
}
```

Important boundary
If a production application needs span-level latency, tokens, tool telemetry or correlation IDs, implement a trace listener or observability adapter; do not expect DecisionEvent alone to be a full distributed trace.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Failure path: invalid model output

A malformed action JSON string fails in ActionOutputParser before an action can execute. A candidate JSON with fewer than two candidates fails `parseCandidates`; unsupported action IDs fail `ActionRegistry.require`; arguments not satisfying the registered definitions fail CoalaAction.validate. The base loop maps an invalid selected action into InvalidModelOutputException when validation is performed on the selected ActionCall.

For research, a bad proposer/evaluator result may be retried by the deliberative procedure up to maxProposalAttempts for DecisionProcedureException cases; exhaustion raises PlanningExhaustedException. For travel, an invalid plan never executes its first step because every step is parsed and validated before `plan.first_steps` is committed. For support, invalid single-action output fails before the mock classification/escalation action runs.

State and observable values

Field	Value / meaning
No action execution?	yes, if parse/validation fails before execute
HTTP behavior	sanitized 502 for invalid model output; some planning errors map to 500
state risk	no real external side effect in these mock adapters

Actual source excerpt: dev/coala/core/action/ActionOutputParser.java (source lines 35-70)

```
        requireOnlyFields(root, Set.of("actionId", "arguments"), "action proposal");
        JsonNode actionId = root.get("actionId");
        if (actionId == null || !actionId.isTextual() || actionId.asText().isBlank()) {
            throw new InvalidModelOutputException("Action JSON must contain a non-empty string 'actionId'.");
        }
        return buildActionCall(actionId.asText(), parseArguments(root.get("arguments")));
    }

    /** Parses a set of at least two distinct alternatives. */
    public List<ActionCall> parseCandidates(String raw, int maxCandidates) {
        if (maxCandidates < 2) throw new IllegalArgumentException("maxCandidates must be at least 2");
        JsonNode root = parseJson(raw, "candidate proposal");
        requireObject(root, "Candidate JSON must be an object.");
        requireOnlyFields(root, Set.of("candidates"), "candidate proposal");
        JsonNode candidates = root.get("candidates");
        if (candidates == null || !candidates.isArray() || candidates.size() < 2) {
            throw new InvalidModelOutputException("Candidate JSON must contain at least two alternatives in a
        'candidates' array.");
        }
        if (candidates.size() > maxCandidates) {
            throw new InvalidModelOutputException("Model proposed " + candidates.size()
                    + " candidates; configured limit is " + maxCandidates + ".");
        }
        List<ActionCall> parsed = new ArrayList<>();
        Set<ActionCall> seen = new HashSet<>();
        for (JsonNode candidate : candidates) {
            requireObject(candidate, "Every candidate must be a JSON object.");
            requireOnlyFields(candidate, Set.of("actionId", "arguments"), "candidate action");
            JsonNode id = candidate.get("actionId");
            if (id == null || !id.isTextual() || id.asText().isBlank()) {
                throw new InvalidModelOutputException("Every candidate must contain a non-empty 'actionId'.");
            }
            ActionCall call = buildActionCall(id.asText(), parseArguments(candidate.get("arguments")));
            if (!seen.add(call)) {
                throw new InvalidModelOutputException("Candidate set contains duplicate action calls; propose
        distinct alternatives.");
            }
            parsed.add(call);
```

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Failure path: action precondition requests replanning

TravelGroundingActions.searchFlights and searchHotels check travel.calendar.checked. If false, they return an observation containing `needs_replan` and explicitly say no provider query was made. PlanFirstDecisionProcedure's default replanCondition detects this token at the start of the next cycle and discards the remaining step list, stores plan.replaced_steps, and requests a revised plan from reasoning.

draft_itinerary similarly returns needs_replan if the calendar, flight inventory, or hotel inventory observations are missing. This is a useful distinction between planning intent and execution reality: an action result can invalidate the previously proposed plan, and the next cycle reacts to the observation rather than blindly continuing.

Actual source excerpt: dev/coala/core/procedure/PlanFirstDecisionProcedure.java (source lines 42-69)

```
@Override protected PlanDecision planNextAction(AgentContext context, ActionRegistry actions) {
    String observation = context.workingMemory().getString("latest_observation", "");
    List<ActionCall> pending = readPending(context.workingMemory().get("plan.first_steps").orElse(null));
    boolean revise = replanCondition.test(observation);
    if (pending.isEmpty() || revise) {
        if (revise && !pending.isEmpty()) context.workingMemory().put("plan.replaced_steps",
    pending.stream().map(PlanFirstDecisionProcedure::state).toList());
        String prompt = "Create a plan of 1 to " + maxSteps + " executable NEXT ACTIONS for the goal, using
    current memory/observations. "
                + "Return strict JSON {\"steps\":[{\"actionId\":\"...\",\"arguments\":{\"key\":\"string
    value\"}}]}. "
                + "Only use the following registered actions and valid argument contracts: " +
    formatter.format(actions.descriptors()) + ". "
                + "A plan is a proposal, not execution. Prefer short plans. JSON only."
                + (revise ? " Previous observation suggests the old plan needs revision: " + observation :
    "");
        List<ActionCall> proposed = parser.parseActionPlan(reasoning.reason("PLAN_FIRST", prompt, context),
    maxSteps);
        for (ActionCall step : proposed) {
            var action = actions.validate(step);
            if (action.kind() != ActionKind.GROUNDING && action.kind() != ActionKind.LEARNING)
                throw new DecisionProcedureException("Plan step must target a registered grounding or
    learning action: " + step.actionId());
        }
        pending = new ArrayList<>(proposed);
        context.workingMemory().put("plan.first_steps",
    pending.stream().map(PlanFirstDecisionProcedure::state).toList());
        context.workingMemory().put("reasoning.latest", null);
    }
    ActionCall selected = pending.remove(0);
    context.workingMemory().put("plan.first_steps",
    pending.stream().map(PlanFirstDecisionProcedure::state).toList());
    context.workingMemory().put("plan.selectedAction", state(selected));
    List<DecisionEvent> events = List.of(
            new DecisionEvent(context.cycle(), DecisionStage.PROPOSAL, "Prepared/reused a plan with " +
    (pending.size() + 1) + " step(s) before selecting the next one."),
            new DecisionEvent(context.cycle(), DecisionStage.SELECTION, "Selected the first remaining plan
    step: " + selected.actionId() + "."));
    return new PlanDecision(selected, List.of(selected), List.of(), events);
```

Important boundary
The successful travel simulation never takes this branch because check_calendar is first and all fixtures return valid observations. The branch is present as a safe recovery path.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

Run it locally and compare the returned trace

Start the application with the deterministic default profile, POST the exact request from page 3, and preserve the returned sessionId. Inspect the run fields: procedure should match this document, cyclesExecuted should be the expected terminal cycle count, terminationReason should be TERMINAL_ACTION, and the finalObservation should match the action result described in the cycle ledger.

For a second request that needs persistent session memory, send the returned sessionId. A made-up session ID is rejected rather than created. For actual OpenAI integration, configure the project's openai profile and API key; that changes the provider behind LanguageModel, not the action catalog or procedure semantics.

Method-call ledger

01	./mvnw -B clean verify (first run the full Maven test suite).
02	./mvnw spring-boot:run (deterministic profile by default).
03	POST /api/use-cases/support-iterative/run with request JSON from page 3.
04	Check returned procedure, cyclesExecuted, terminationReason, finalObservation and events.
05	Save returned sessionId if you want to resume that demo session.

CUSTOMER SUPPORT TRIAGE - ITERATIVE REASON-ACT-OBSERVE / SOURCE-LEVEL EXECUTION TRACE

What the guide proved and what remains outside the simulation

The successful trace proves the source-code path for this fixture: how request fields become a run request, how the service assembles the procedure and action allow-list, how each planning method returns structured action data, how the base loop validates and executes the selected registered action, how the observation becomes next-cycle state, and how terminal=true returns AgentRunResult.

It does not prove correctness of an actual LLM’s reasoning, current web facts, real calendar access, real travel availability, airline/hotel booking, account security, ticket creation, durable memory after restart, or production authorization. Those require real adapters plus integration/security tests. The simulated provider must be labeled as simulated until replaced and verified.

Important boundary
The implementation described in this report is the supplied source snapshot. If the code changes, regenerate the walkthroughs so line references and output fixtures remain accurate.